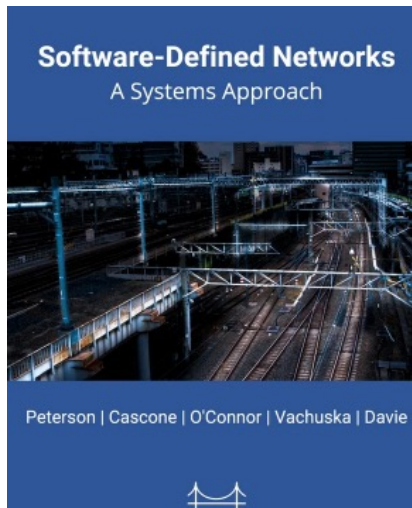


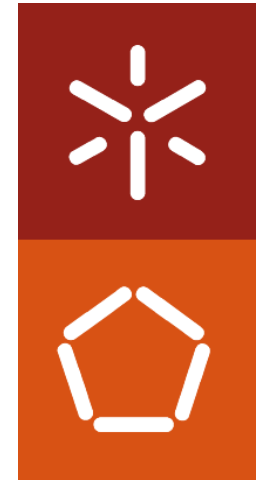
Software Defined Networks (SDN)

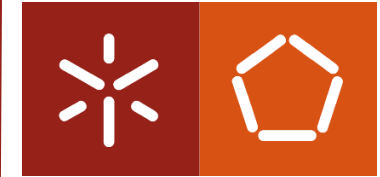
Gestão e Virtualização de Redes

1º Semestre



<https://sdn.systemsapproach.org/>

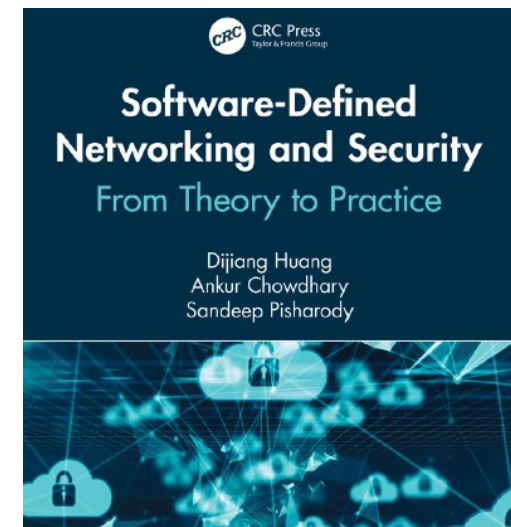


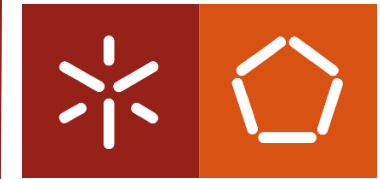


Conceitos

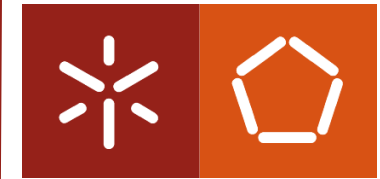
Baseado nos seguintes materiais:

- Preambulo da Parte 1 do livro
“Software-Defined Networking and Security”,
Dijiang Huang, Ankur Chowdhary, Sandeep Pisharody, CRC Press

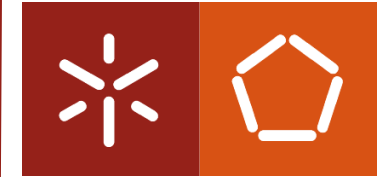




- **Definido por software (*software-defined*)**
 - capacidade de abstração da gestão e administração da tecnologia
- **Redes definidas por software**
 - Capacidade de controlar o provisionamento de equipamentos de rede, LANs Virtuais, regras de firewall, qualidade de serviço (QoS) etc. usando software
- **Um sistema definido por software confia no software para suportar **Abstração, Automação, Ajustamento (AAA)****
 - Abstração dos recursos físicos – usando APIs que facilitam a gestão e alocação
 - Automação de ações / controlo – ações e controlos podem ser executados com base numa lógica de aplicação, mediante condições ou com base em eventos
 - Ajustamento de configurações (reconfiguração) – ajuste dos recursos através de configuração preditiva



- **Software-Defined Networking (SDN)**
 - Arquitetura de rede emergente, baseada na separação do plano de dados e do plano de controlo
- **Software-Defined Storage (SDS)**
 - gestão do armazenamento de dados por software, independente do armazenamento físico (permite agregação de recursos de armazenamento e gestão de uma pool de recursos partilhados)
- **Software-Defined Data Centers (SDDC)**
 - Centro de dados totalmente virtualizado e oferecido como um serviço
- **Software-Defined Power (SDP)**
 - Camada de abstração para gerir o consumo de energia, movendo as aplicações para centros de dados com melhores condições
- **Software-Defined Vehicle (SDV)**
 - Gestão das funcionalidades de um veículo por software
- **Software-Defined Everything (SDE)**
 - Generalização



- **Software-Defined Virtual Networking (SDVN)**

- Conceito que reúne as duas tecnologias principais: SDN e NFV

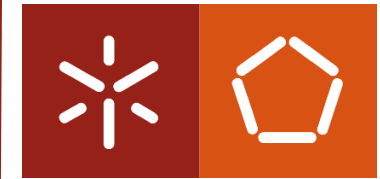
NFV - Network Function Virtualization

arquitetura de rede virtual baseada em soluções de virtualização que virtualizam classes de funções dos nós da rede em componentes que se podem conectar ou encadear para criar serviços de comunicação e de rede

SDN - Software-defined Networking

arquitetura de rede baseada na separação do plano de controlo e de dados, onde as entidades de rede (routers, switches, firewalls, etc) são instanciadas e controladas de forma automatizada, recorrendo ao uso de controladores.

SDN e NFV Complementam-se: enquanto o NFV move serviços do hardware especializado para um ambiente virtual, o SDN acrescenta a inteligência que automatiza processos de controlo



Parte 1 – Visão Geral

Baseado nos seguintes materiais:

- Capítulo 1 do livro “Software-Defined Networks: A Systems Approach”, Peterson, Cascone, O’Connor, Vachuska, and Davie, Grátis online: <https://sdn.systemsapproach.org/index.html>
- Alguns slides e/ou imagens de Jennifer Rexford
(<https://www.cs.princeton.edu/courses/archive/spr12/cos461/docs/lec24-sdn.pdf>)

O que leva ao SDN?



- **Internet – A Rede das Redes**

- Uma Infraestrutura global de sucesso
- Aplicações nos sistemas finais (*hosts*), rede de comutação de pacotes com serviço de entrega baseado no melhor esforço, ao qual foram sendo adicionados mecanismos de QoS
- Inovação tremenda nas aplicações:
 - Web, Streaming online, Redes Sociais, VoIP e Videoconferência, IoT
- Mas...

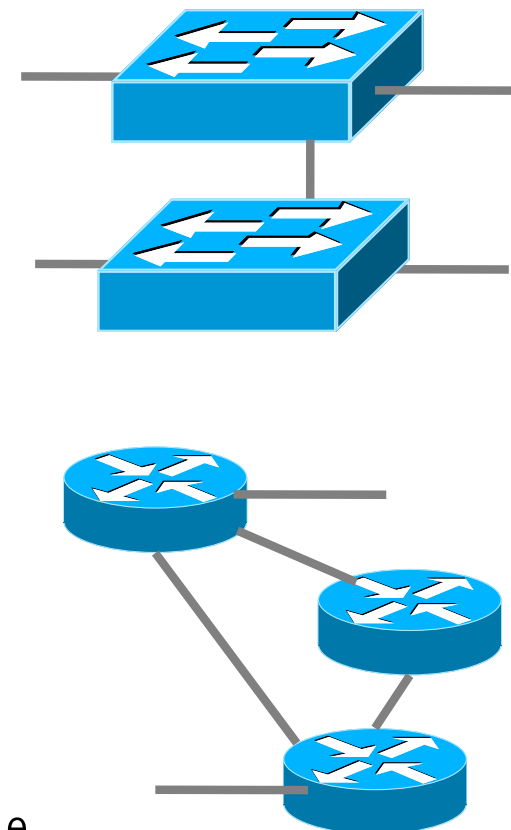
A inovação é fácil na periferia da rede
mas **muito difícil** no núcleo da rede!....

O que leva ao SDN?



- **Equipamento muito específico e “fechado”**

- Apenas corre *firmware* e *software* do fabricante
- Com interfaces proprietárias e específicas por fabricante
- Ciclo de evolução muito lento:
 - Os protocolos são primeiro normalizados, nos comités de normalização e só depois implementados e disponibilizados nos equipamentos
 - Só equipas muito restritas podem trabalhar no software
 - Poucos podem propor e implementar novas soluções
- Ciclo de correção de problemas é também lento!
 - Quando é detetado um “*bug*”, tem de se comunicar primeiro ao fabricante, que o reproduz, corrige, cria *patch* ao software e lança uma nova *release* ou um “*security fix*”, aos clientes com licença!



Impacto na segurança e no custo, também no desempenho e robustez



O que leva ao SDN?

- **A gestão da rede é uma tarefa árdua**
 - Muito jargão, uma panóplia enorme de protocolos, muitos tipos de tráfego, de pacotes, etc..
 - Uma imensidão de equipamentos com diferentes características, diferentes fabricantes, sistemas operativos, APIs, etc
- **Operar uma rede custa muito dinheiro**
 - Mais de metade dos custos da rede são custos de operação!
 - Quando há um erro qualquer da operação, há prejuízos significativos
- **Enorme probabilidade de “bugs” no software dos equipamentos**
 - São milhões de linhas de código
 - As falhas e vulnerabilidades provocam falhas em cascata

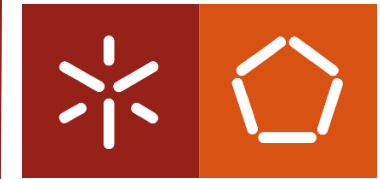
O que leva ao SDN?



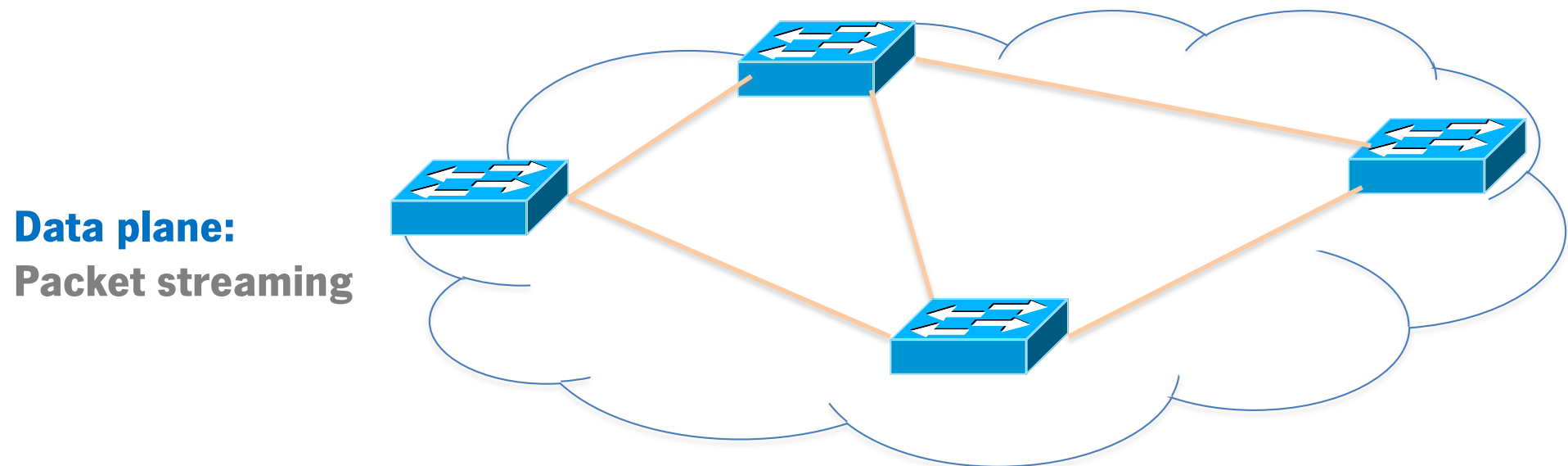
- **A rede está no centro de tudo!**
 - Nas empresas, em centros de dados, em casa, etc
 - Uma falha da rede tem impacto em todas as atividades

Necessidade de criar e gerir redes de forma mais ágil e confiável

SDN – Visão geral

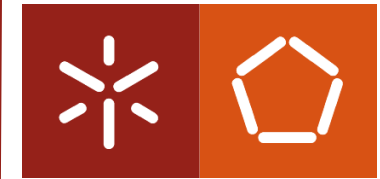


- **As redes "tradicionais"**



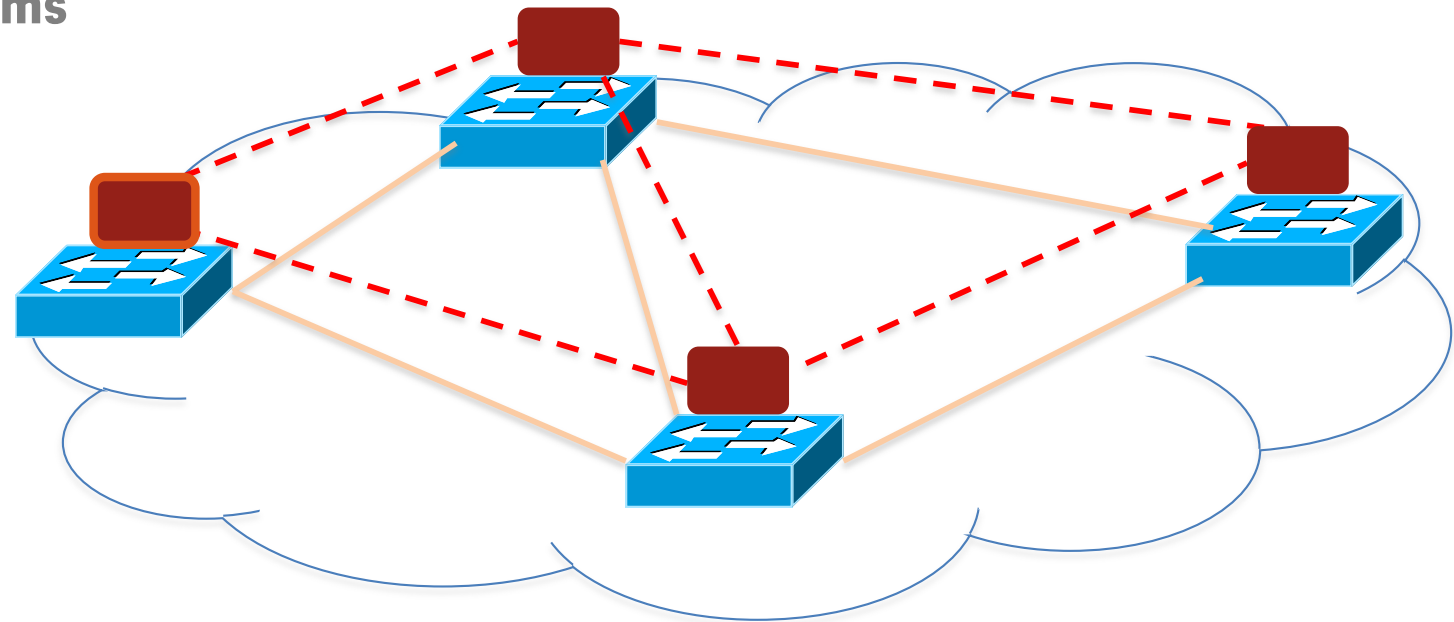
**Forward, filter, buffer, mark,
rate-limit, and measure packets**

SDN – Visão geral

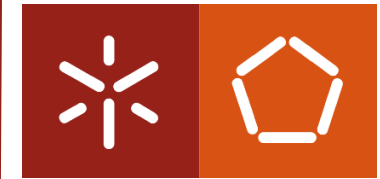


- **As redes “tradicionais”**

Control plane:
Distributed algorithms

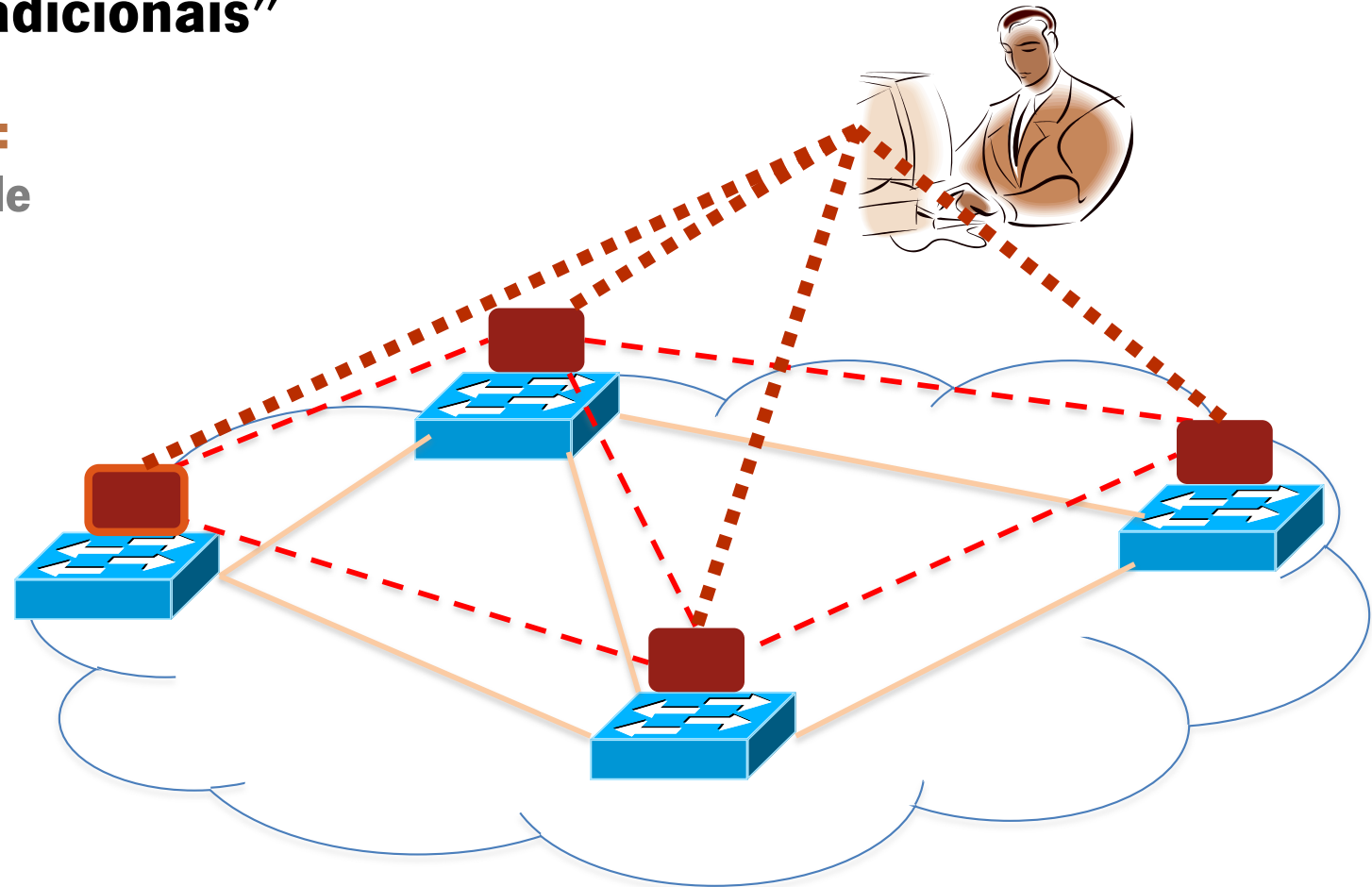


SDN – Visão Geral



- **As redes “tradicionais”**

Management plane:
Human time scale

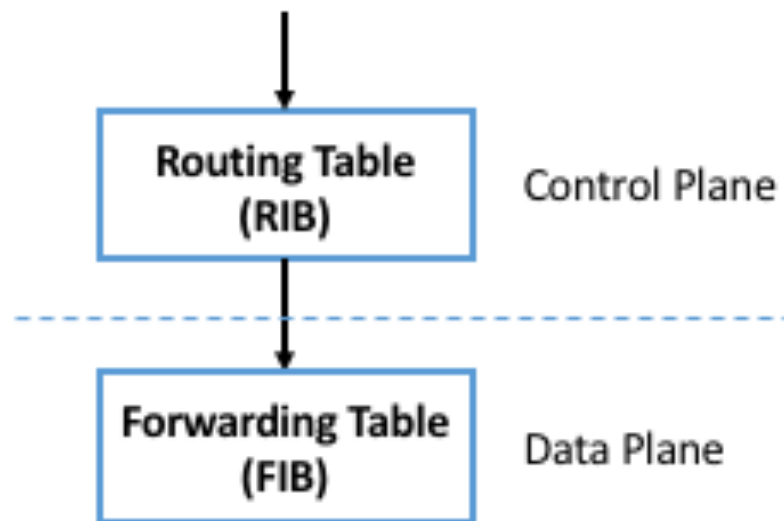


Collect measurements and configure the equipment

SDN – Visão Geral



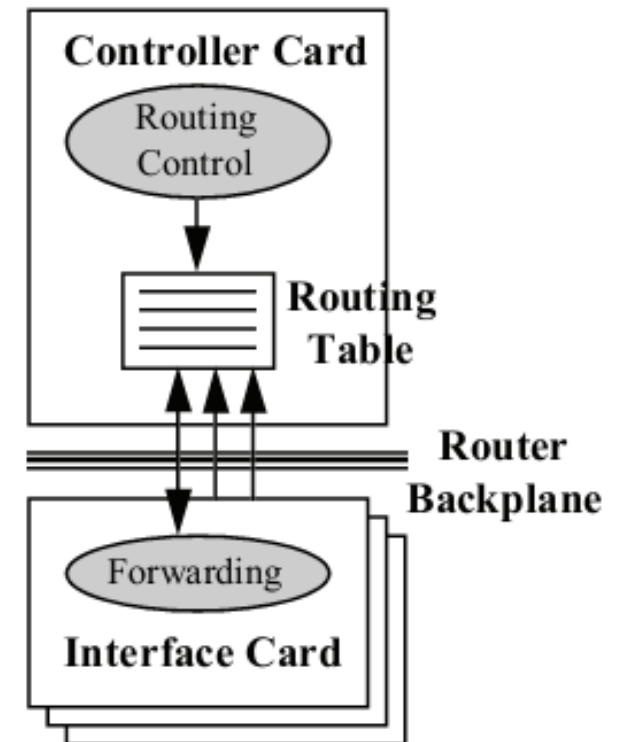
- **Plano de dados e Plano de controlo**



<https://sdn.systemsapproach.org/intro.html>

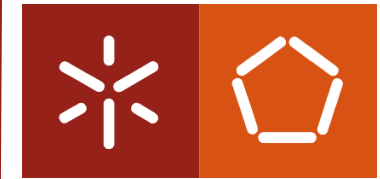
Plano de Controlo
(Encaminhamento)

Plano de dados
(Reenvio)

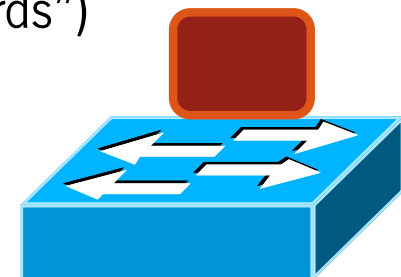


James Aweya, IP router architectures: an overview

SDN – Visão Geral



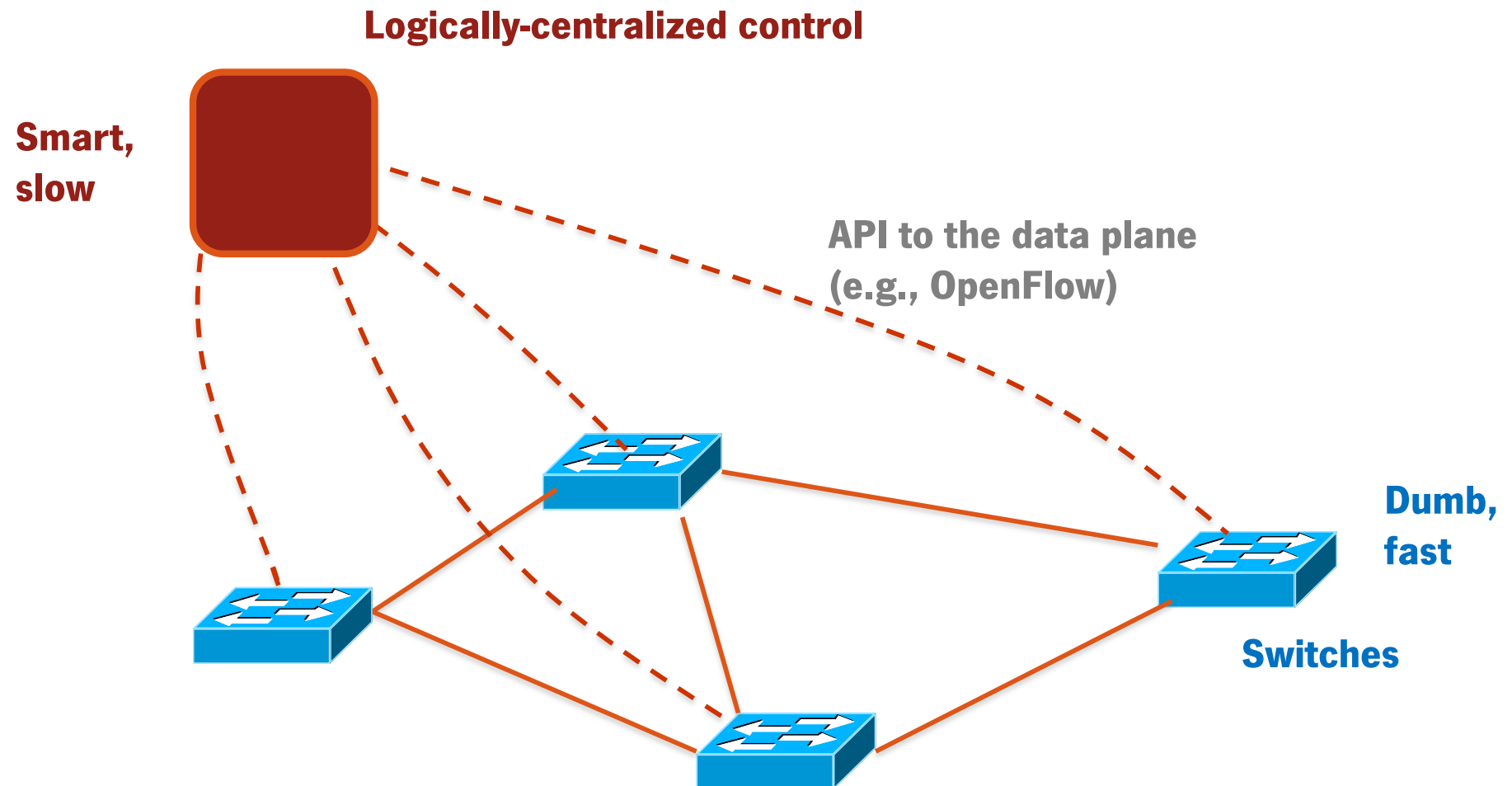
- E se “retirássemos” o plano de controlo dos equipamentos? O que se ganha com isso?
 - Facilidade de **gestão** da rede
 - Plano de controlo e plano de gestão mais próximos do utilizador
 - Não é necessário “inverter” as operações do plano de controlo
 - Um percurso mais fácil e rápido para a **inovação**
 - Menor dependência dos vendedores de equipamentos e das próprios processos de normalização dos protocolos
 - Mais fácil testar ideias em fase pré-standard
 - **Interoperabilidade** fácil de manter (com os “standards”)
 - Equipamentos mais **baratos** e mais **simples**!
 - Software minimalista e à medida



SDN – Visão Geral



- **Redes “SDN”**



SDN: Plano de dados



- **Regras** simples para processar o **envio de pacotes**
- **Quatro componentes fundamentais nas regras**
 - *Padrão de filtragem (**Pattern**)*: análise aos campos dos cabeçalhos dos pacotes
 - *Ações (**Actions**)*: *drop, forward, modify, envia* ao controlador...
 - *Prioridade (**Priority**)*: resolve as ambiguidades quando as regras se sobrepõem
 - *Contadores (**Counters**)*: *#bytes e #packets*

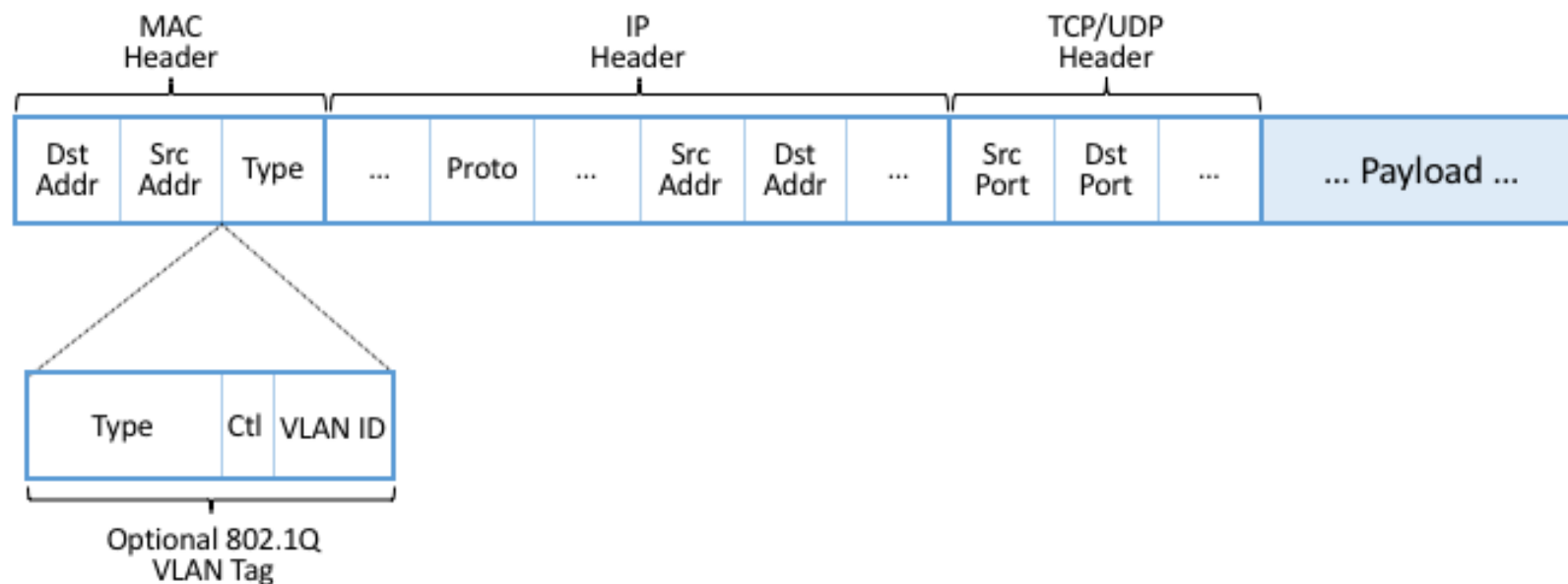


1. ***src=1.2.*.*, dest=3.4.5.* → drop***
2. ***src = *.*.*, dest=3.4.*.* → forward(2)***
3. ***src=10.1.2.3, dest=*.*.* → send to controller***

SDN: Plano de dados

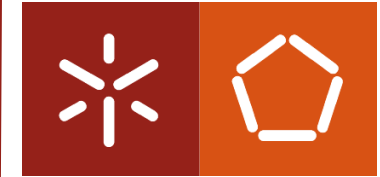


- Campos considerados na norma **OpenFlow** original para as regras da tabela de fluxos



- Figura: campos de cabeçalho correspondentes na especificação OpenFlow original
- Uma regra de fluxo é um par *Match-Action*: qualquer pacote que corresponda à primeira parte da regra deve ter a ação associada aplicada a ele
- Originalmente as ações incluíam “encaminhar pacote para uma ou mais portas” e “descartar pacote”, além de “enviar pacote para o controlador” como regra de escape!

SDN: Plano de dados



- Permite diferentes funcionalidades, unificando diferentes equipamentos:

ROUTER (*layer 3*)

- *Match*: prefixo IP de destino mais longo
- *Action*: *forward* por uma porta de saída

SWITCH (*layer 2*)

- *Match*: endereço MAC de destino
- *Action*: *forward* par uma porta ou *flood* para todas as portas

FIREWALL

- *Match*: endereços IP e portas TCP/UDP
- *Action*: permitir ou não (*permit* ou *deny*)

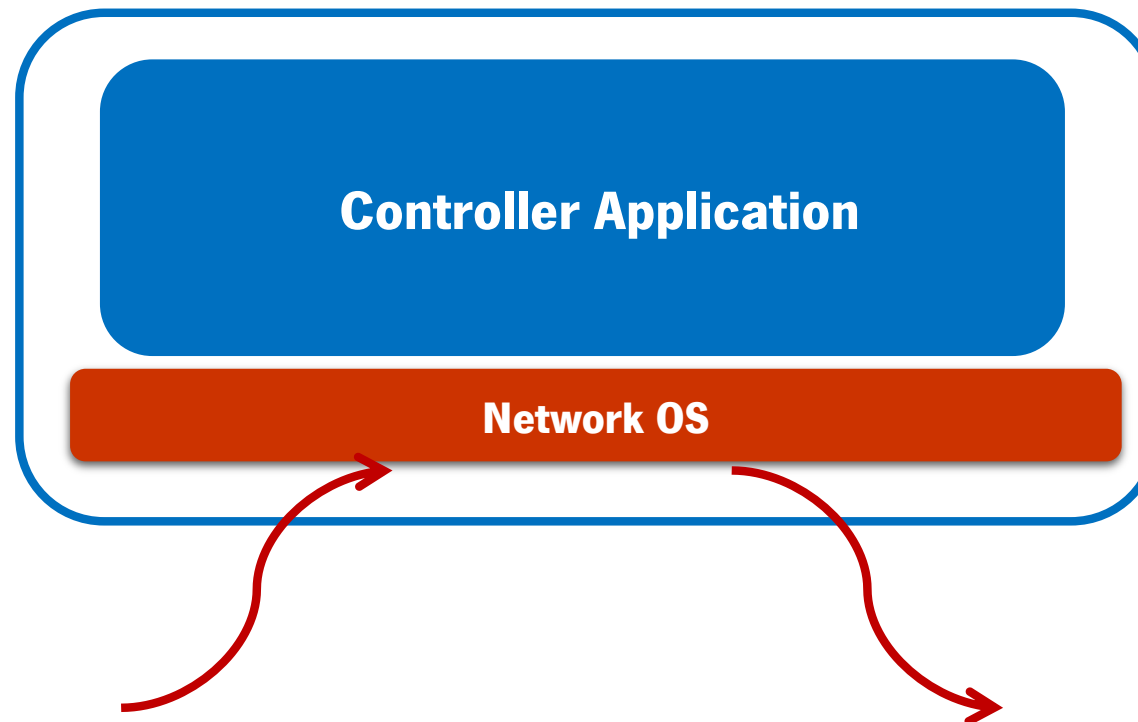
NAT

- *Match*: endereço IP e porta
- *Action*: substitui endereço IP e porta (*rewrite*)

SDN: Plano de controlo



- Permite a “**Programabilidade**” da rede



Eventos vindos dos switches

Alterações da topologia,
Estatísticas de tráfego,
Chegada de pacotes

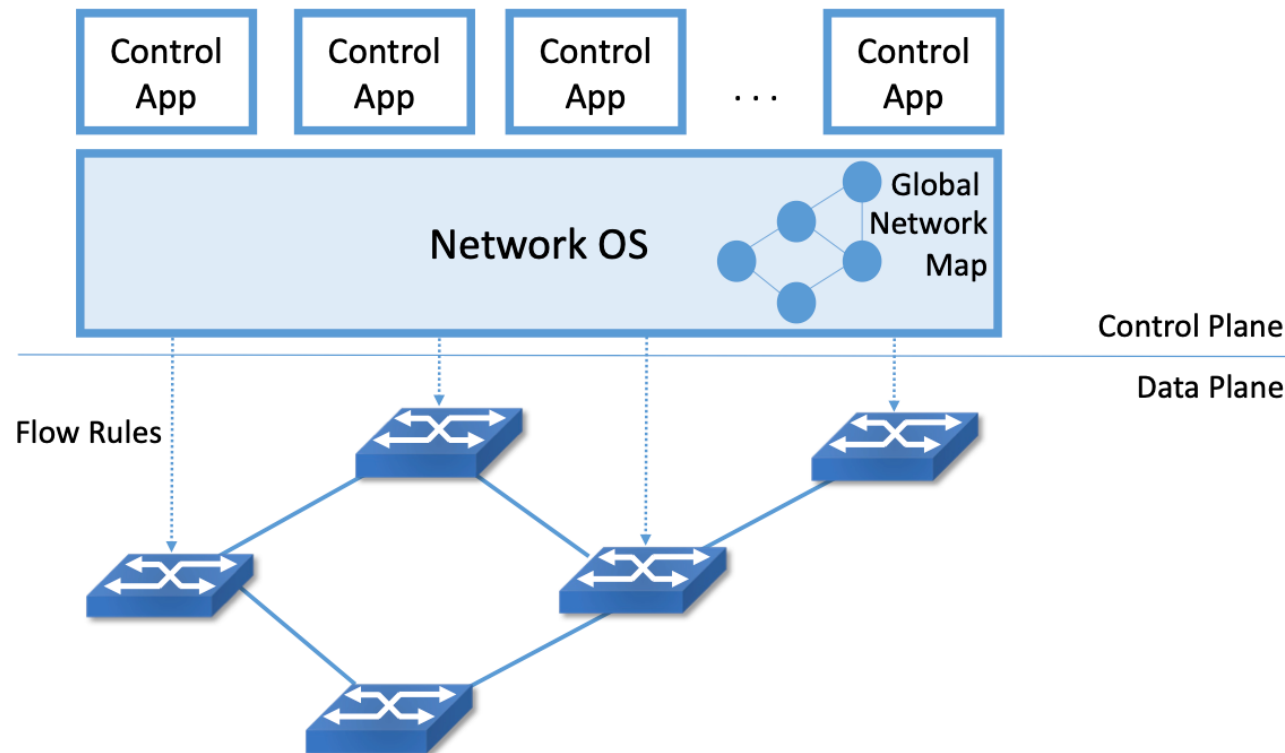
Comandos enviados para os switches

Instalação/remoção de regras,
Pedido de estatísticas (query),
Envio de pacotes

SDN: Plano de controlo



- **Network Operating System (NOS)**

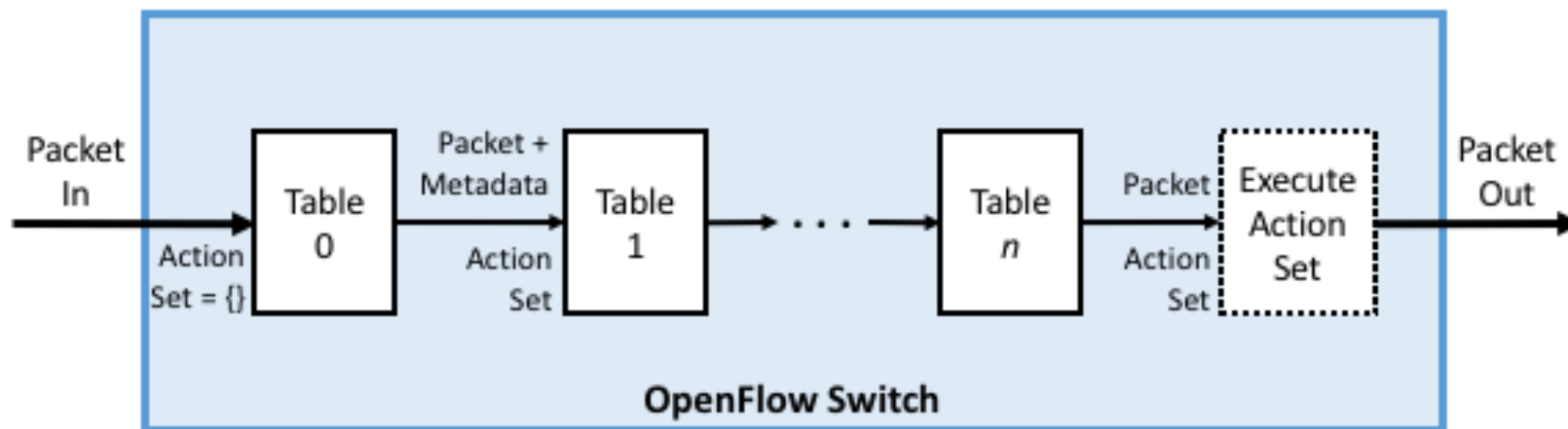


- NOS: suporta um conjunto de aplicações de controlo e fornece um ponto de controlo logicamente centralizado para um plano de dados de rede subjacente
- Um passo à frente em relação à simples desagregação do plano de dados e controlo

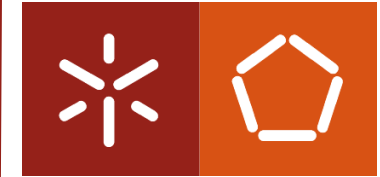
SDN: Plano de dados



- **OpenFlow Switch**



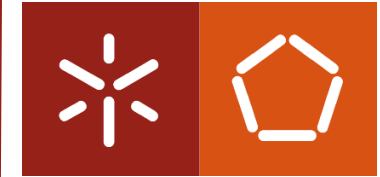
- Em vez de uma única tabela de fluxos, uma sequência de tabelas (*forwarding pipeline*)
- Os *switches* realmente implementam uma série de tabelas de fluxo, cada uma focada num subconjunto dos campos de cabeçalho
- Programação do plano de dados!



Parte 2 – Casos de uso

Baseado nos seguintes materiais:

- Capítulo 2 do livro “Software-Defined Networks: A Systems Approach”, Peterson, Cascone, O’Connor, Vachuska, and Davie <https://sdn.systemsapproach.org/index.html>
- Alguns slides e/ou imagens de Jennifer Rexford
(<https://www.cs.princeton.edu/courses/archive/spr12/cos461/docs/lec24-sdn.pdf>)



- **Exemplos de aplicações**

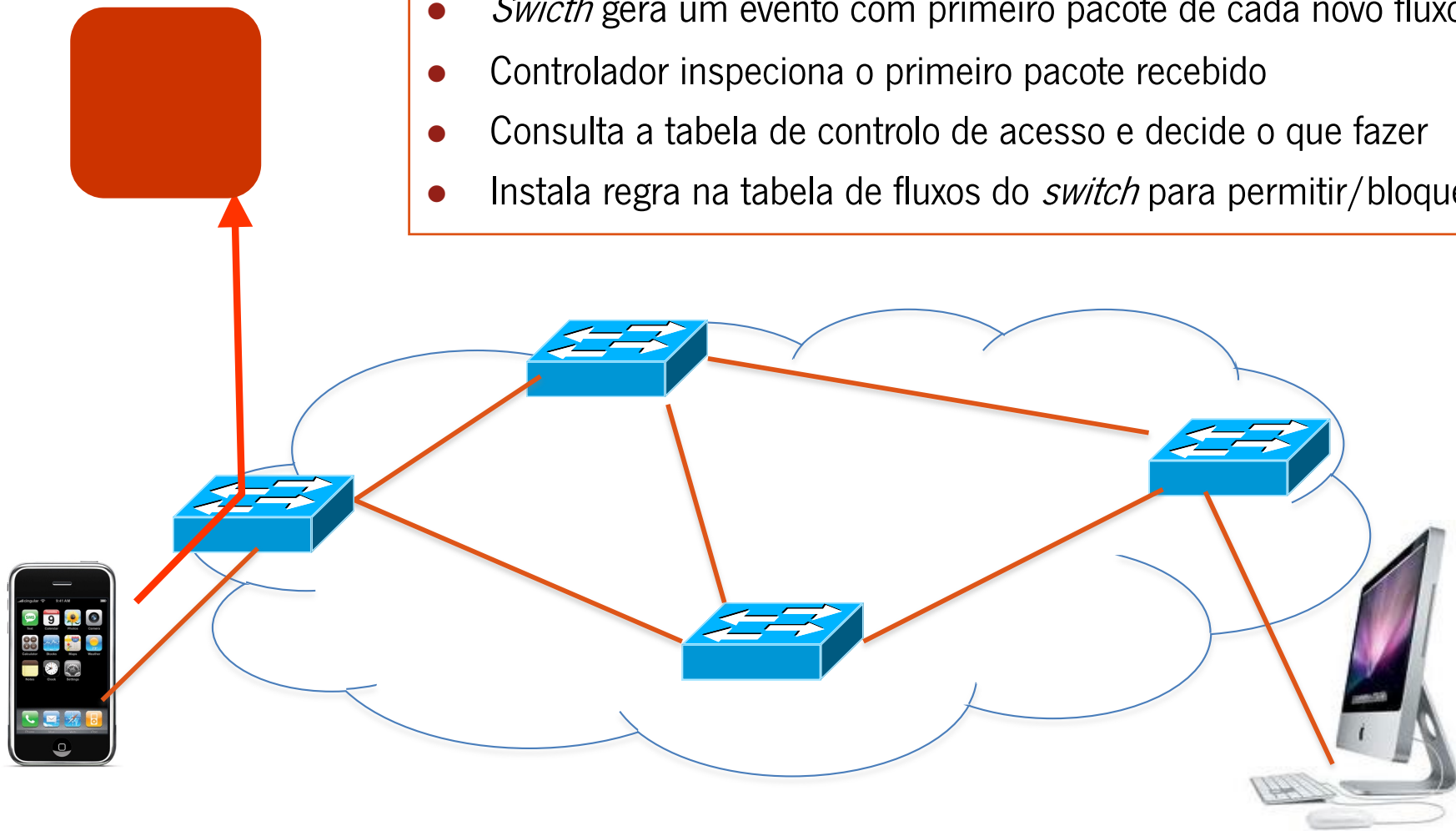
- Controlo de acessos dinâmico (por fluxo!) (*dynamic access control*)
- Mobilidade sem descontinuidade (*seamless mobility*)
- Balanceamento de carga dos servidores (*load balancing*)
- Virtualização da rede (*Network virtualization*)
- Fábricas de comutação (*switch fabric*)
- Redes alargadas definidas por software (SD-WAN)
- Telemetria da rede (*Network telemetry*)
- Detecção de ataques de negação de serviço
-

SDN: Casos de uso



- **Controlo de acessos dinâmico (por fluxo!)**

- *Switch* gera um evento com primeiro pacote de cada novo fluxo
- Controlador inspeciona o primeiro pacote recebido
- Consulta a tabela de controlo de acesso e decide o que fazer
- Instala regra na tabela de fluxos do *switch* para permitir/bloquear



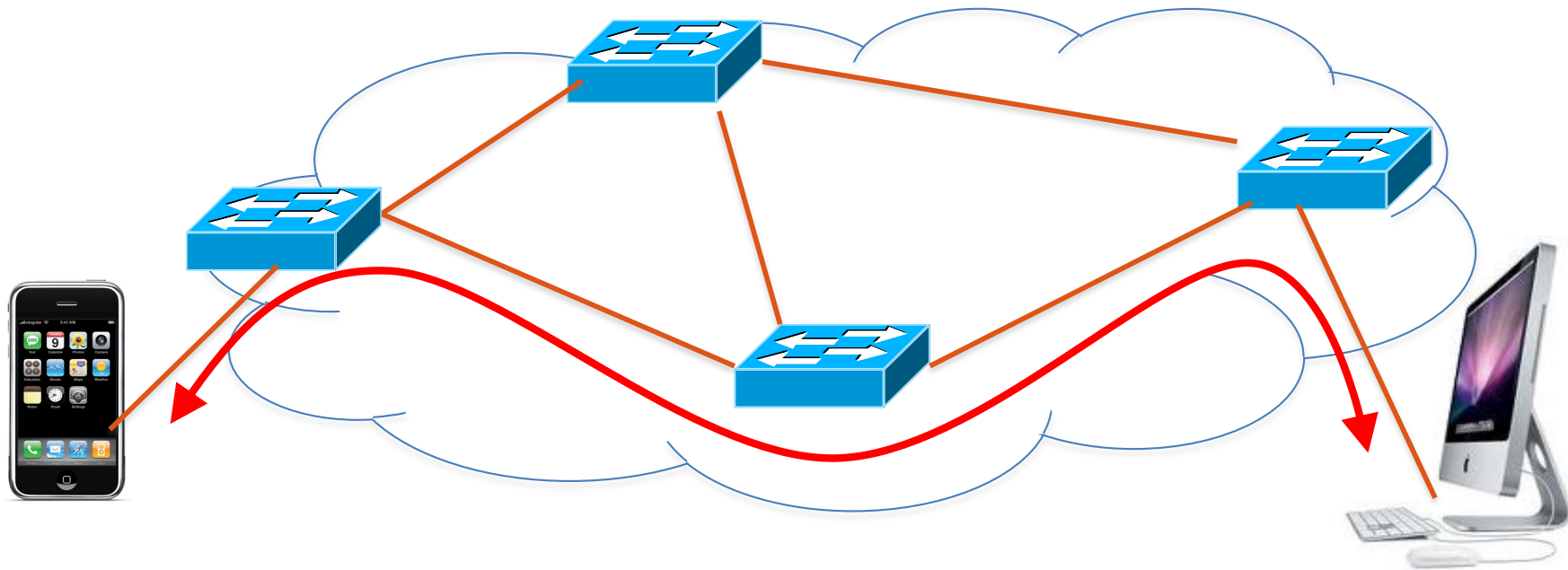
SDN: Casos de uso



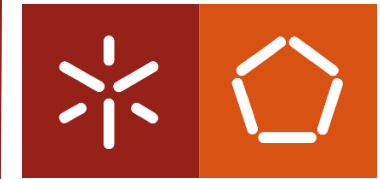
- **Controlo de acessos dinâmico (por fluxo!)**



- *Switch* gera um evento com primeiro pacote de cada novo fluxo
- Controlador inspeciona o primeiro pacote recebido
- Consulta a tabela de controlo de acesso e decide o que fazer
- Instala regra na tabela de fluxos do *switch* para permitir/bloquear



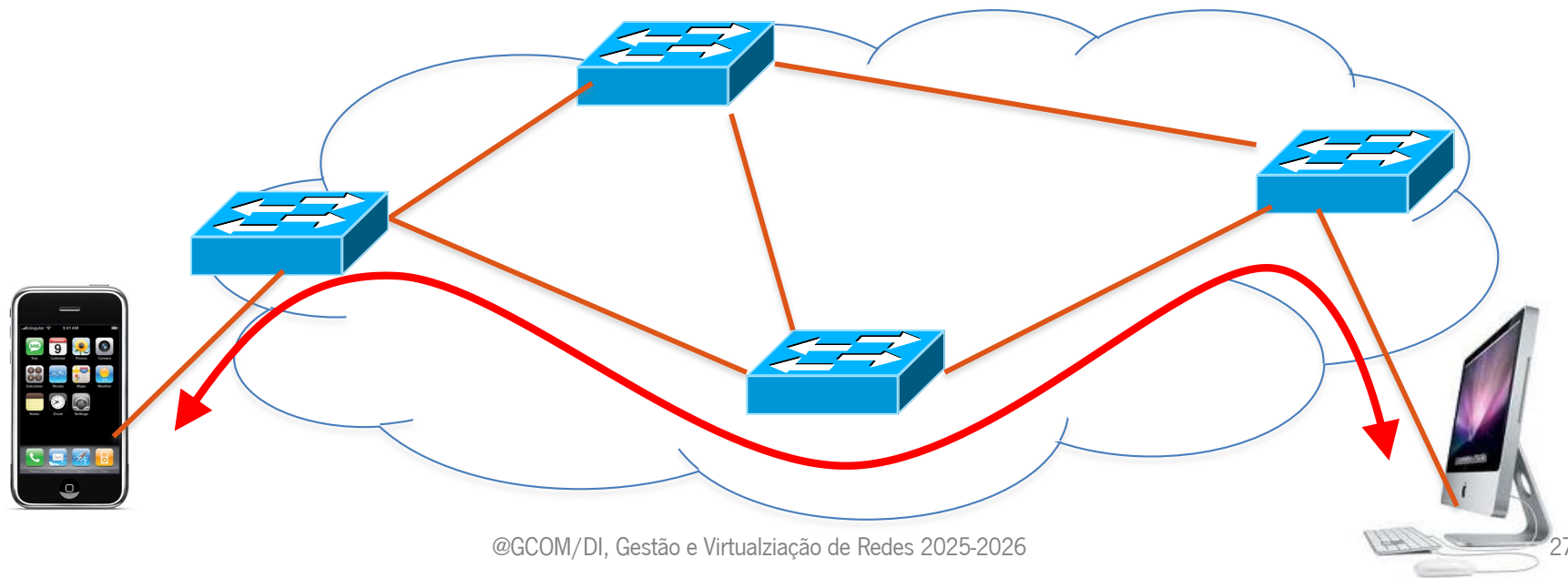
SDN: Casos de uso



- **Mobilidade sem descontinuidade**



- Terminal móvel com fluxo ativo move-se para outra localização



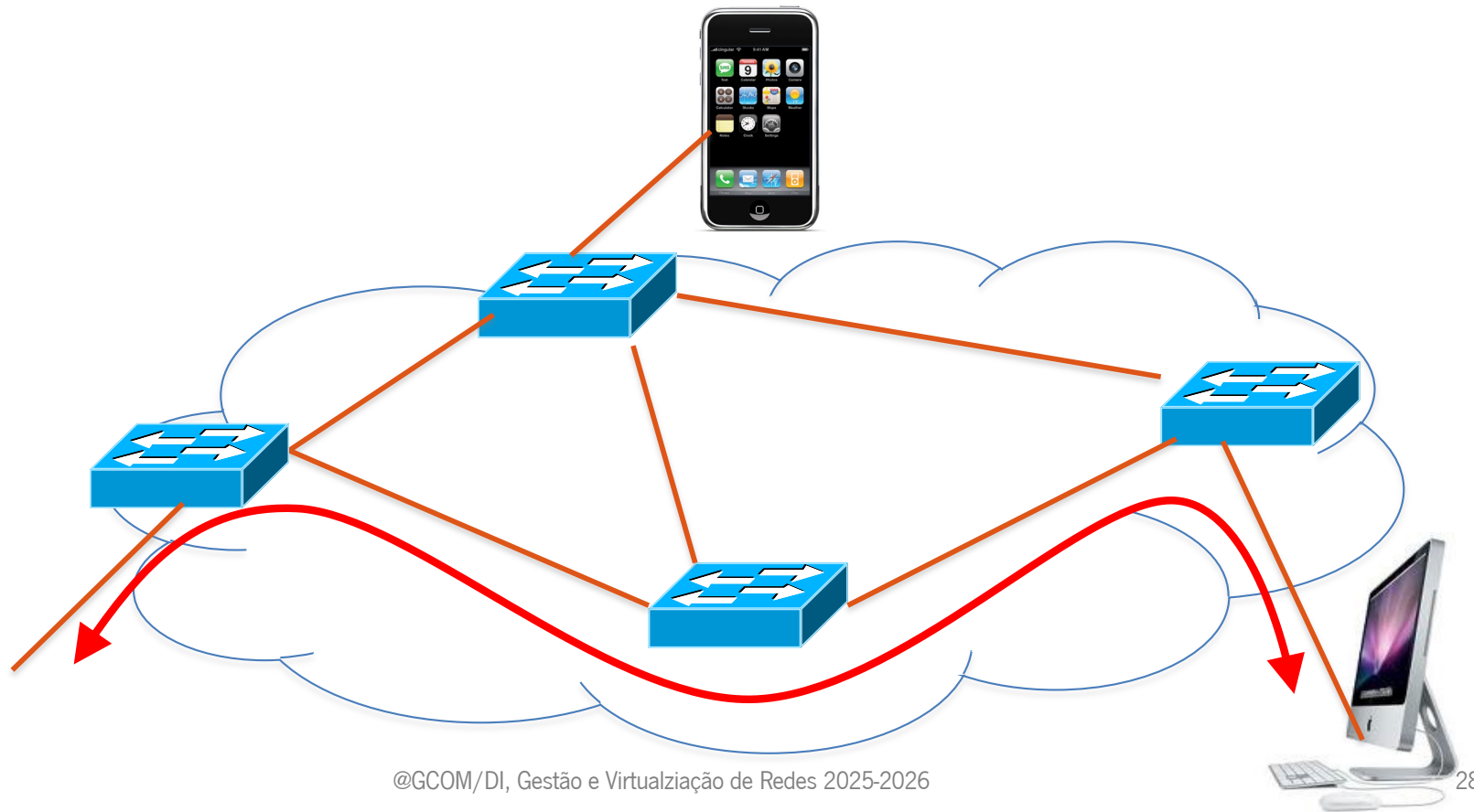
SDN: Casos de uso



- **Mobilidade sem descontinuidade**



- Terminal móvel com fluxo ativo move-se para outra localização



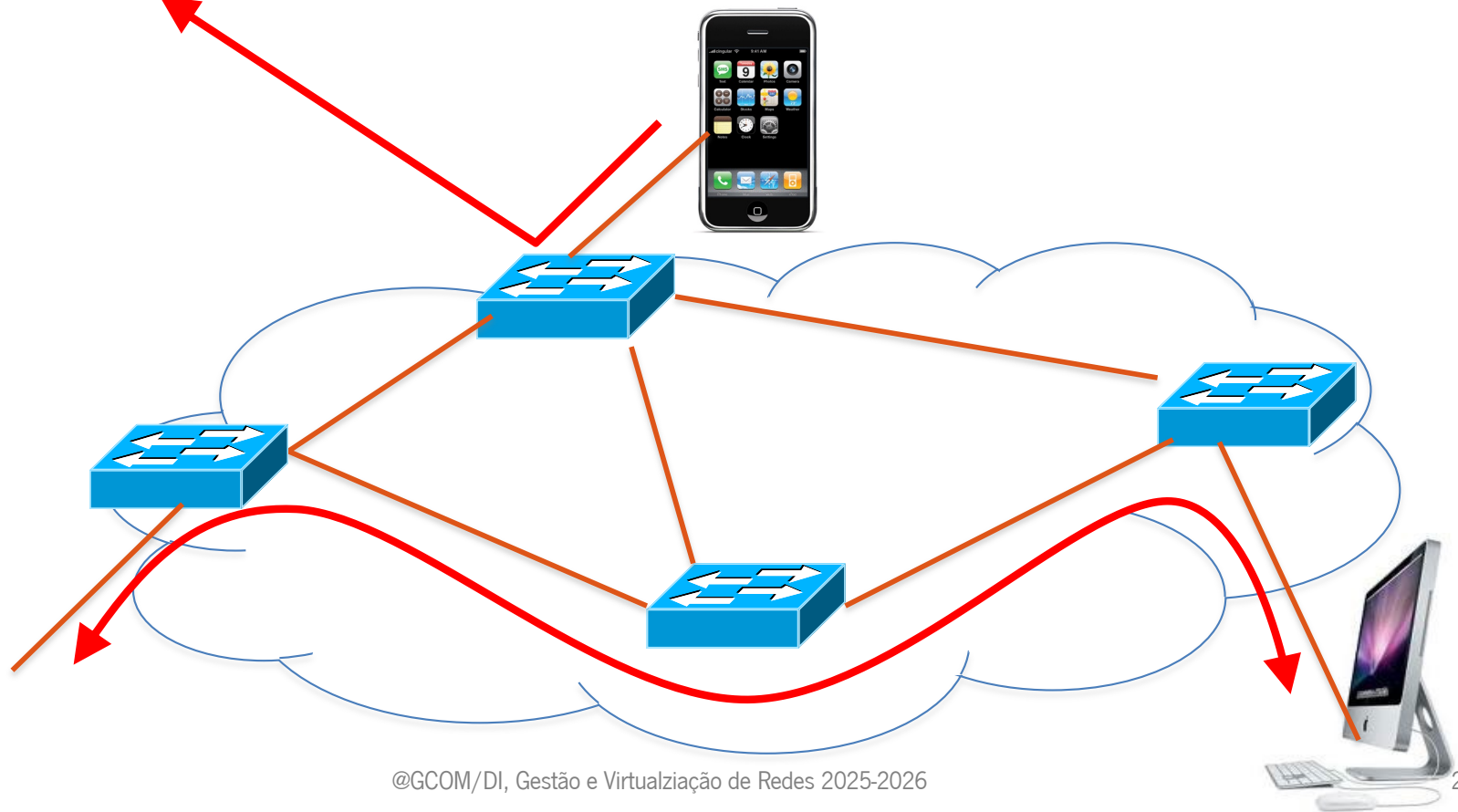
SDN: Casos de uso



- **Mobilidade sem descontinuidade**



- Controlador recebe um evento na nova localização do terminal
- Modifica as regras para instalar uma nova rota



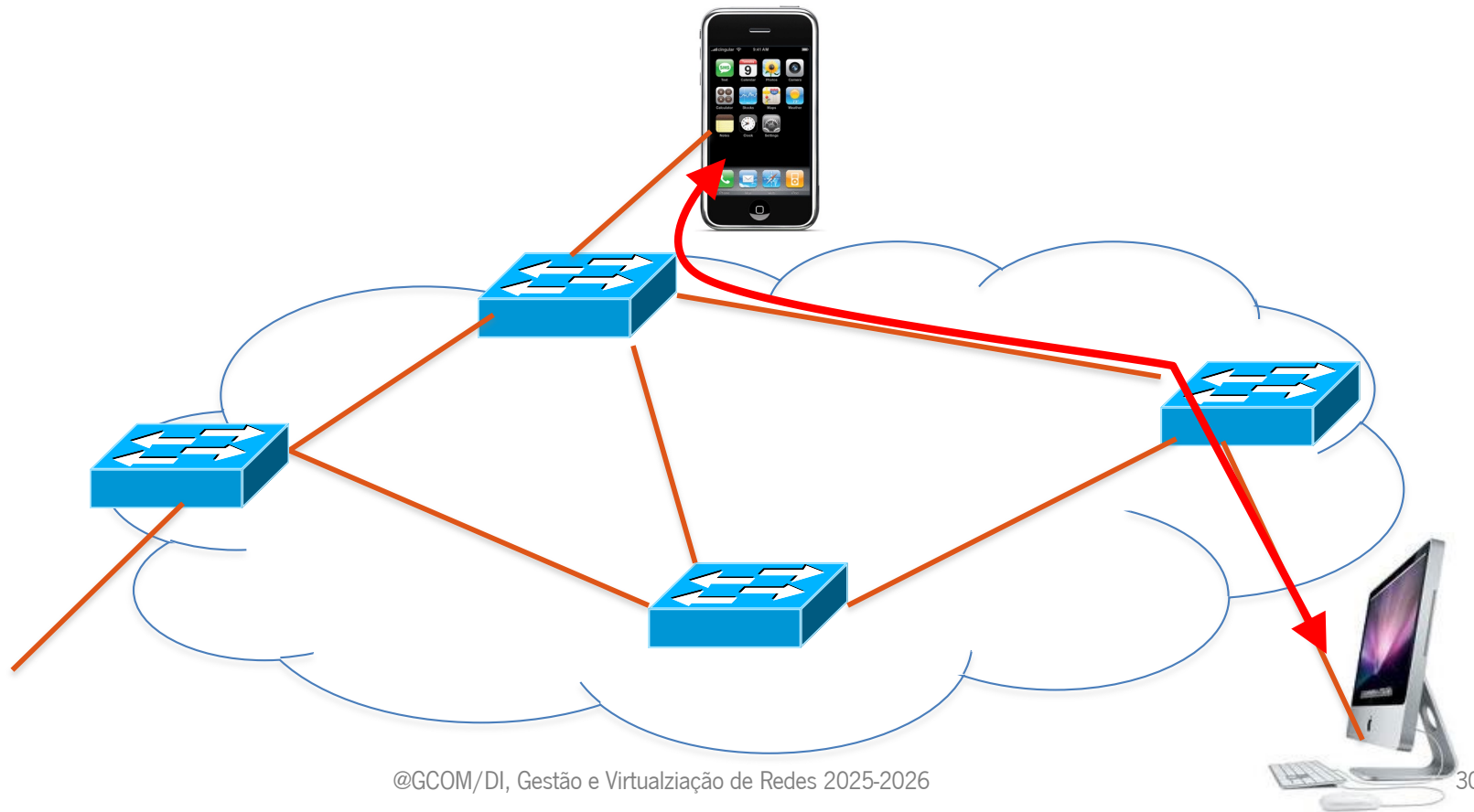
SDN: Casos de uso



- **Mobilidade sem descontinuidade**



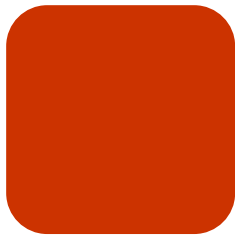
- Controlador recebe um evento na nova localização do terminal
- Modifica as regras para instalar uma nova rota



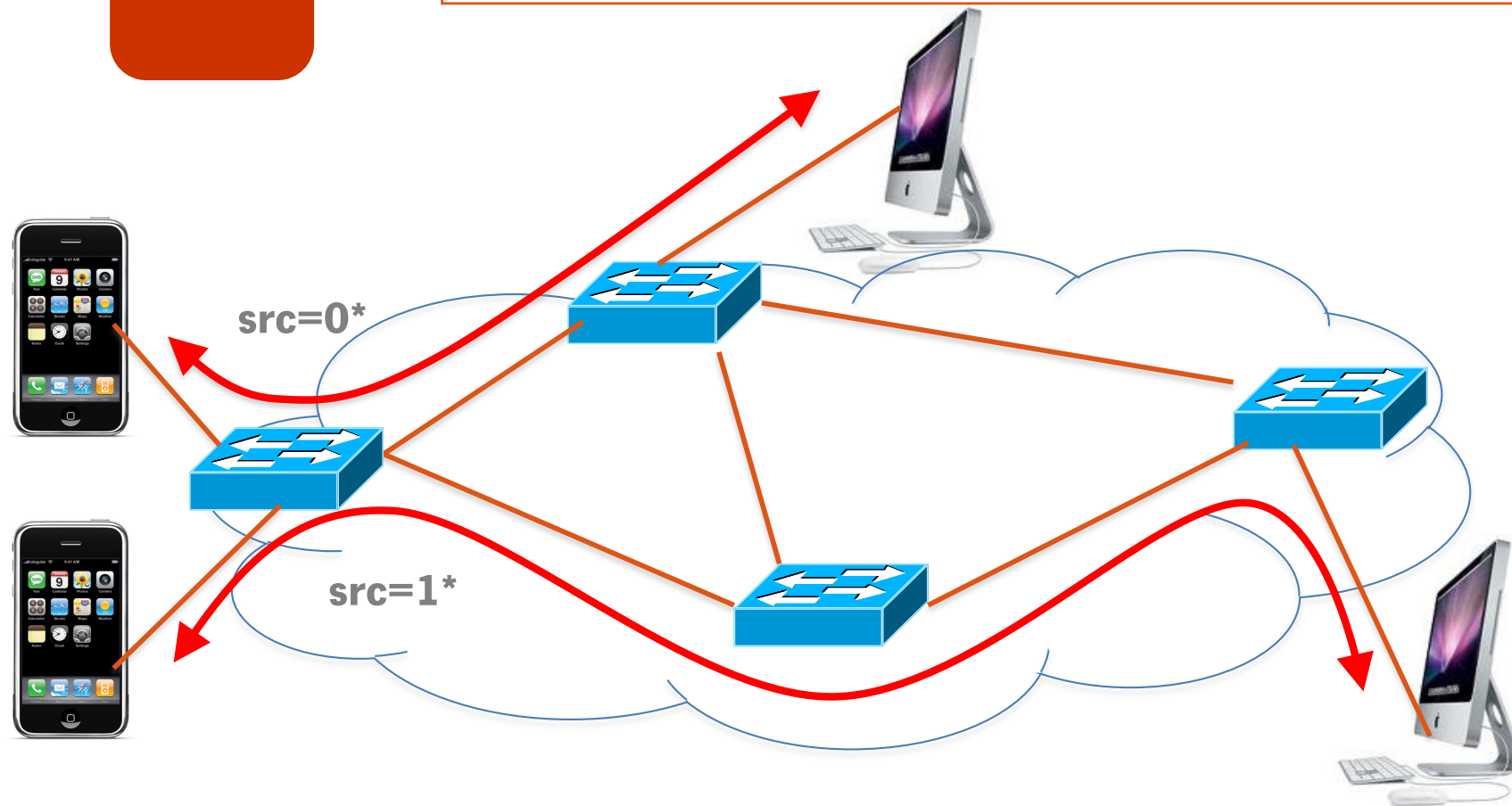
SDN: Casos de uso



- **Balanceamento de carga**



- Controlador instala, de forma proativa ou reativa, rotas distintas de balanceamento de carga em função do endereço de origem



SDN: Casos de uso



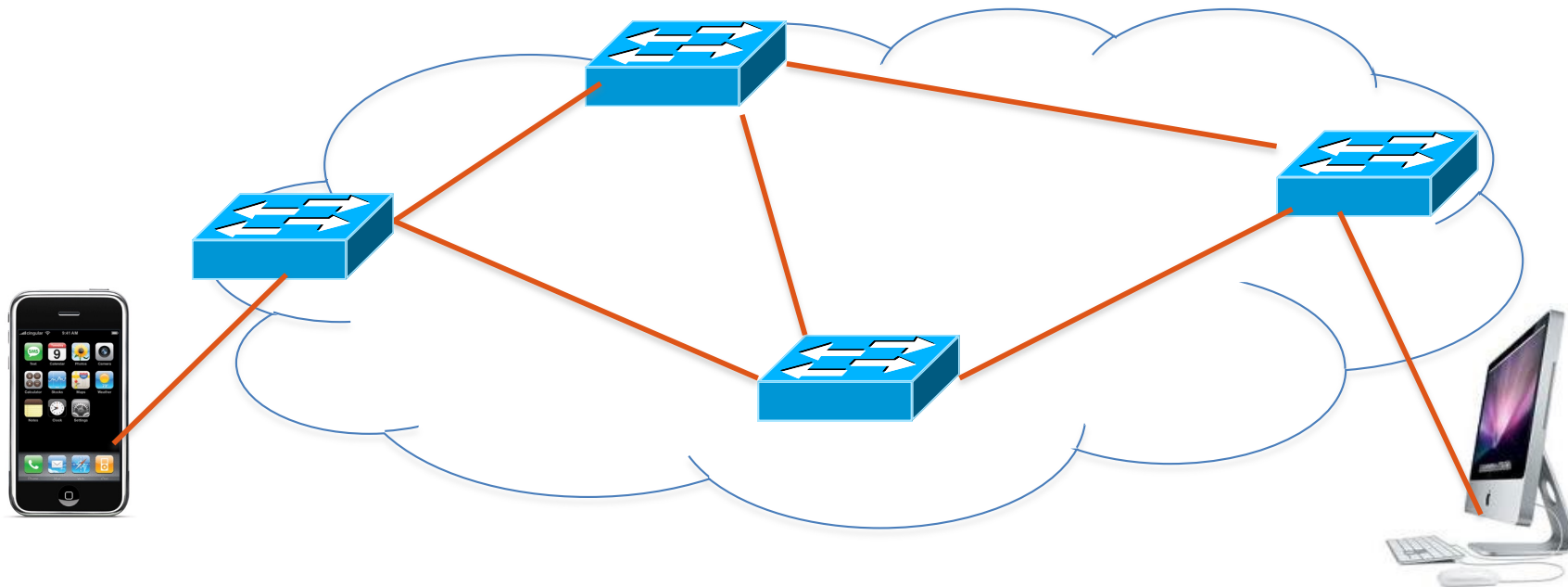
- **Virtualização da rede**

Controlador #1

Controlador #2

Controlador #3

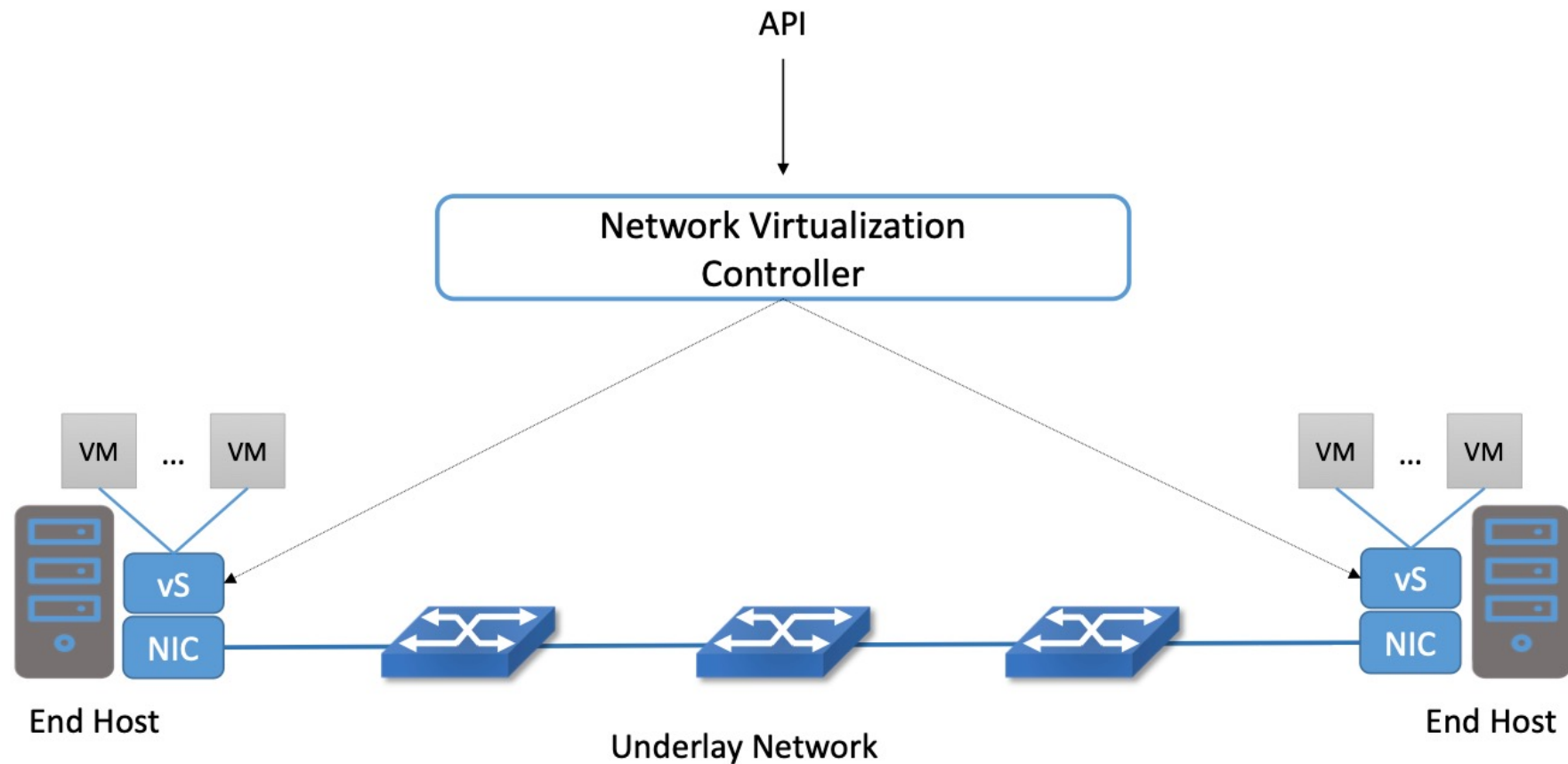
Partição da rede em função dos cabeçalhos dos pacotes



SDN: Casos de uso



- **Virtualização da rede**

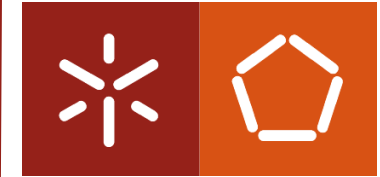




- **Virtualização da rede**

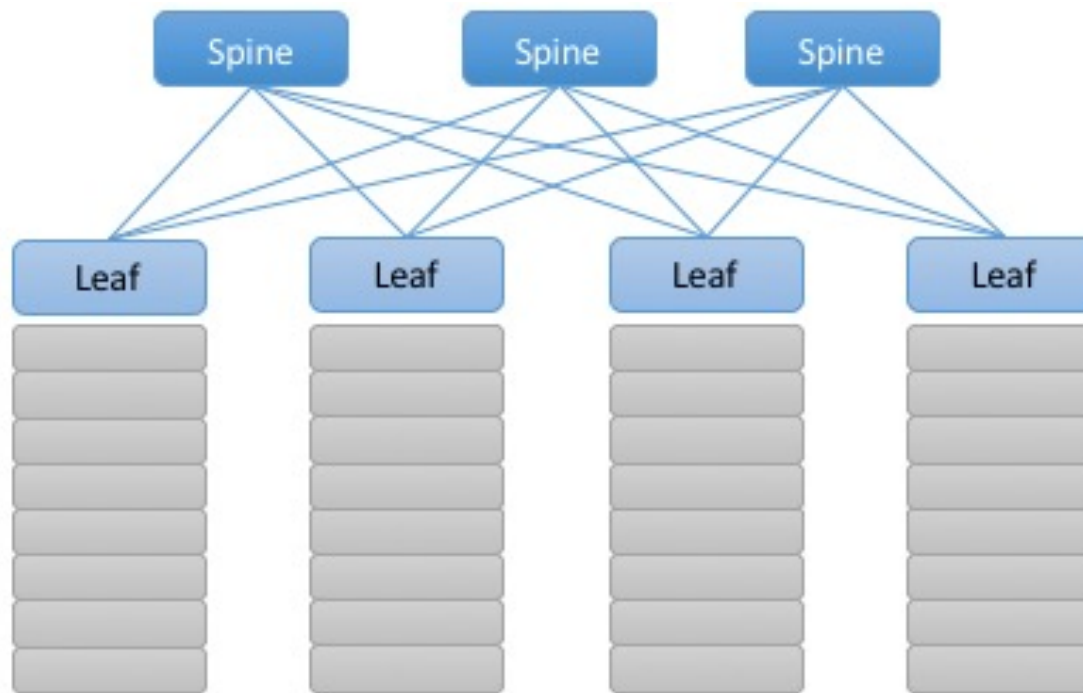
- Provavelmente um dos primeiros grandes casos de sucesso SDN
- A ideia é usar o controlador SDN para criar, manter, gerir e destruir redes virtuais:
 - VLAN (*Virtual Local Area Networks*) e também VPN (*Virtual Private Networks*)
 - Sem intervenção manual do administrador
- Papel importante na virtualização dos servidores (computacional)
 - Automatiza a migração de máquinas virtuais, mantendo o endereço IP e a conectividade
 - E também das versões *light* dos *containers*, como *Kubernetes*
- Vantagens:
 - Uso versátil do encapsulamento VXLAN e outros
 - Micro-segmentação da rede com redução da superfície de ataques à segurança

SDN: Casos de uso



- **Fábricas de comutação (*switching fabrics*)**

- Talvez o caso de uso mais predominante e mais determinante

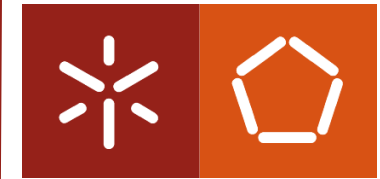


A espinha dorsal

...e as folhas

- Num centro de dados, cada bastidor (rack) tem um *switch* designado por **ToR** (*Top of the Rack*), que interliga todos os servidores nesse bastidor (*sw-folha*)

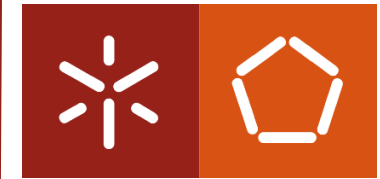
SDN: Casos de uso



- **Fábricas de comutação (*switching fabrics*)**

- Na verdade existem tipicamente dois ***switches ToR*** para garantir redundância!
- Os **ToR**, também designados ***switches leaf*** (*folha*) ligam-se de forma redundante aos *switches spine* (*espinha*)
- Cada ***switch leaf*** conecta-se a um subconjunto de ***switches spine***, com os seguintes requisitos:
 1. Tem de haver múltiplos caminhos entre cada par de bastidores
 2. Cada caminho entre bastidores (*rack-to-rack*) tem dois saltos de distância (ou seja, usa um único *switch* intermédio da espinha dorsal)

Qualquer caminho entre dois servidores dentro do centro de dados está ou a **dois** saltos (*server-leaf-server*) ou a **quatro** saltos (*server-leaf-spine-leaf-server*)



- **Fábricas de comutação (*switching fabrics*)**

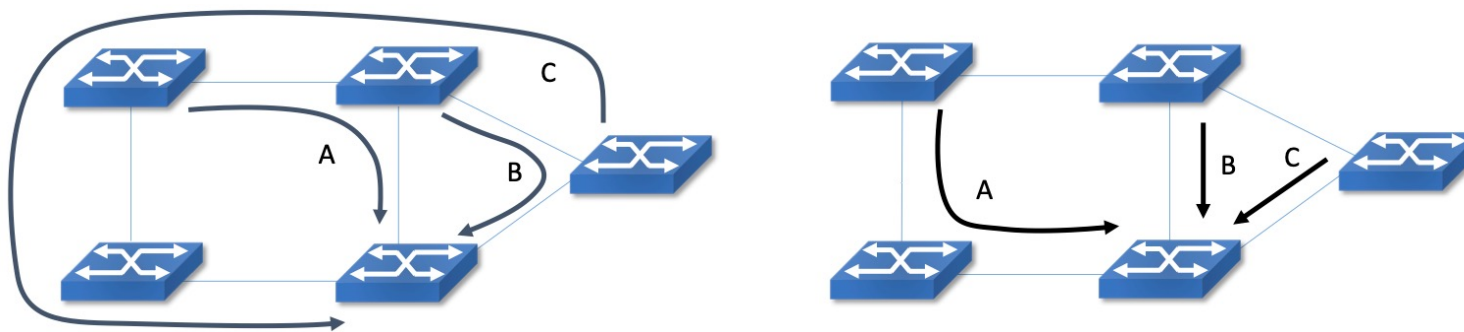
- O controlador SDN normalmente estabelece *forwarding* de nível 2 (*L2-switching*) dentro do bastidor e *routing* de nível 3 (*L3-routing*) até aos ToR
- Como há dois caminhos possíveis pode-se usar *Equal-Cost Multipath (ECMP)* *forwarding* para balancear o tráfego
 - Um *round-robin* simples dos fluxos por cada um dos dois caminhos disponíveis, que aproveite a largura de banda disponível.
- Internamente o controlador usa conceitos de comutação de etiquetas (*label-switching*) parecidos com os usados no MPLS.

Este caso de uso vai ser estudado com mais detalhe, dada a sua relevância



● Engenharia de tráfego para redes WAN

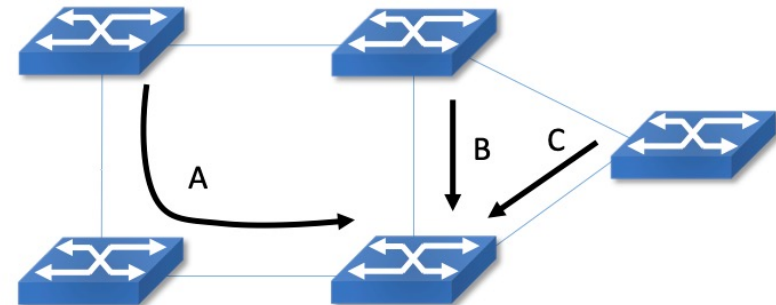
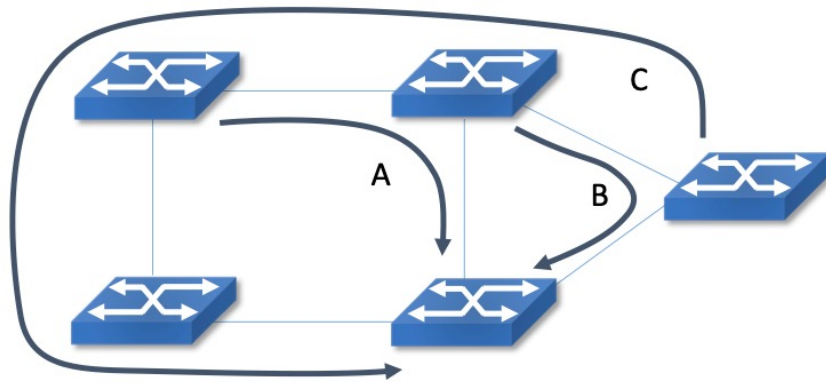
- Mais um caso de uso inspirado na *cloud*: engenharia de tráfego aplicado à rede alargada de interligação entre *datacenters*
- A Google já descreveu publicamente o seu *backbone* privado, designado por B4, construído apenas por *switches bare-metal* e controladores SDN
- A Microsoft fez o mesmo e falou do seu sistema SWAN
- Tanto no B4 como no SWAN, há uma aplicação de controlo, que faz engenharia de tráfego, gere os recursos de acordo com as necessidades das aplicações!
- O modelo SDN de controlo centralizado permite fazer isso



SDN: Casos de uso



- **Engenharia de tráfego para redes WAN**



- Na figura, assumindo que existem três fluxos (A, B, C) em simultâneo e que todos os *links* possuem a mesma capacidade, um algoritmo possível seria:
 - Pegar no primeiro fluxo A, e colocá-lo no caminho mais curto
 - Pegar no segundo fluxo B, e dar-lhe o segundo caminho mais curto, porque o primeiro já está a suportar o fluxo A
 - Finalmente, o fluxo C tem de ficar no caminho mais longo
- No entanto, um controlador SDN, com visão centralizada da rede, poderia tratar a alocação dos três fluxos de uma vez, distribuindo melhor (imagem da direita)!



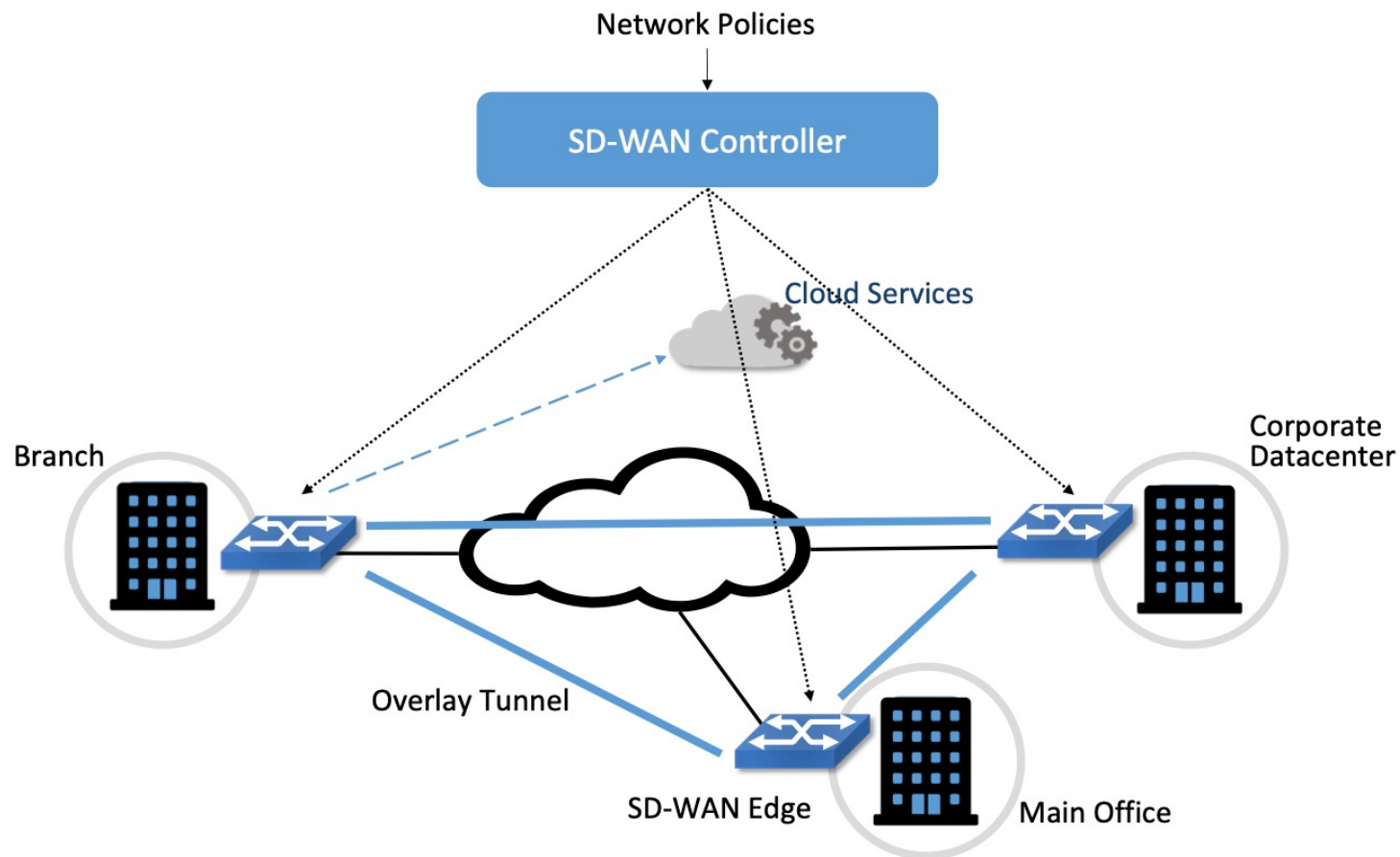
- **Engenharia de tráfego para redes WAN**

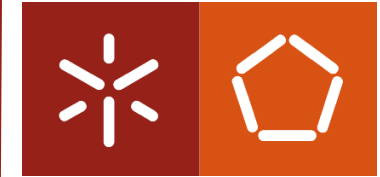
- Embora seja um exemplo forçado, a alocação sub-ótima só se consegue evitar com controlo centralizado
- Quando há alterações, por exemplo uma falha de um link, o controlador reorganiza e otimiza o mapa de tráfego na rede
- Com os anos, os algoritmos foram ficando mais sofisticados:
 - Em vez de se limitarem a fazer balanceamento de tráfego equilibrado por caminhos de igual custo, passaram a considerar “classes de serviço”
 - Exemplos de classes de tráfego consideradas pelo B4 da Google:
 - Cópia de dados dos utilizadores para outro datacenter (emails, documentos áudio/video)
 - Acesso a espaço em disco remoto por sistemas de computação distribuída
 - Mover quantidades gigantes de dados para sincronização de estado entre datacenters
 - Qual o mais crítico?
- **Centralizando a decisão e programando a rede**, a Google consegue utilizações próximas dos 100% da capacidade (comparadas com os anteriores 30 a 40%)

SDN: Casos de uso



- **Redes alargadas definidas por software (SD-WAN)**





- **Redes alargadas definidas por software (SD-WAN)**

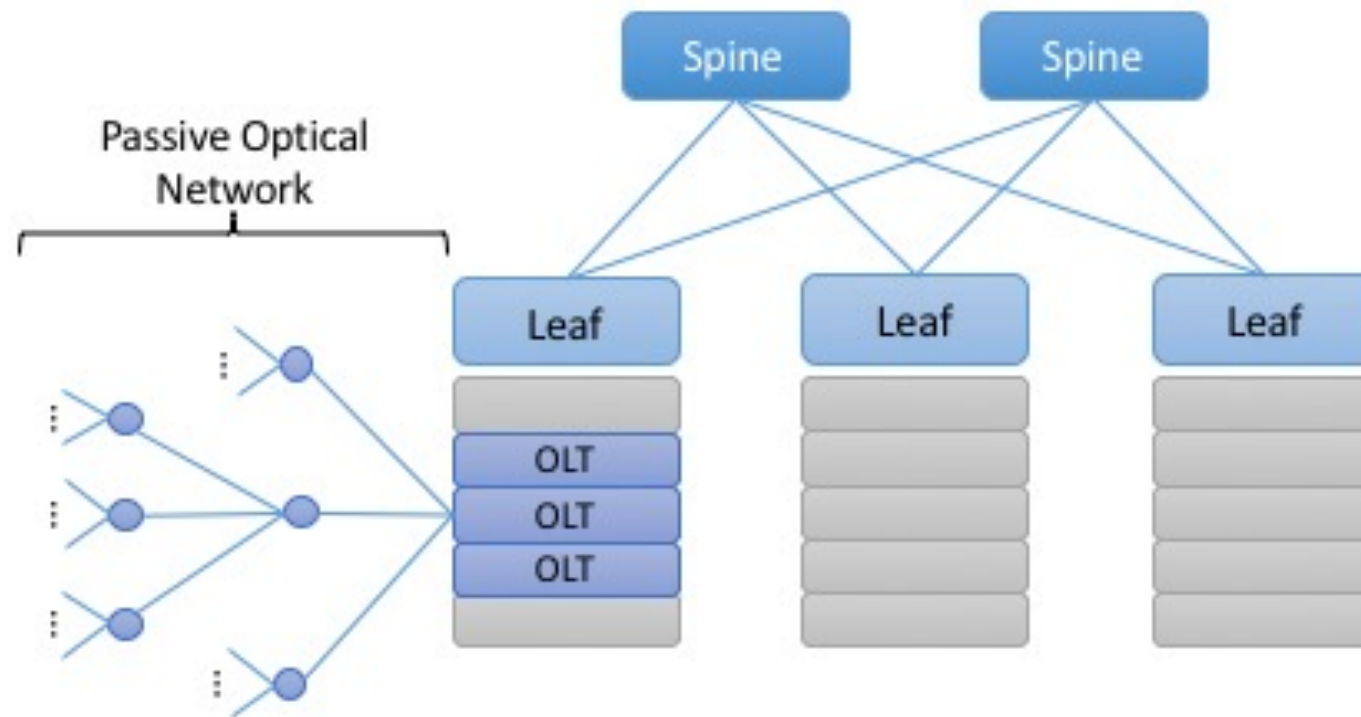
- Ambiente empresarial: a abordagem mais usada tem sido o MPLS (séc. XXI)
- Mas o SD-WAN tem surgido como uma alternativa ao MPLS-BGP:
 - Estabelecer redes privadas (VPN) é o objetivo
 - A empresa pretende gestão centralizada, as mesmas políticas de acesso e de segurança, as mesmas regras de prioridade no tráfego, etc
 - Em vez de configuração manual, centralizada, as empresas pretendem provisionamento *"zero-touch"*
- O objetivo número 1 é poder definir políticas centrais que sejam aplicadas em todos os locais da empresa, estejam onde estiverem:
 - *"Colocar o tráfego YouTube com a mais baixa prioridade"*
 - *"Permitir acesso a este serviço cloud a todos os escritórios"*
 - Etc..

SDN: Casos de uso



- **Redes de acesso**

- Dois exemplos: **Passive Optical Networks (PON)** e **Radio Access Network (RAN)**
- Exemplos de redes que tradicionalmente usam hardware específico! Mas...
 - O objetivo é também usar hardware genérico (*bare-metal*) e SDN...





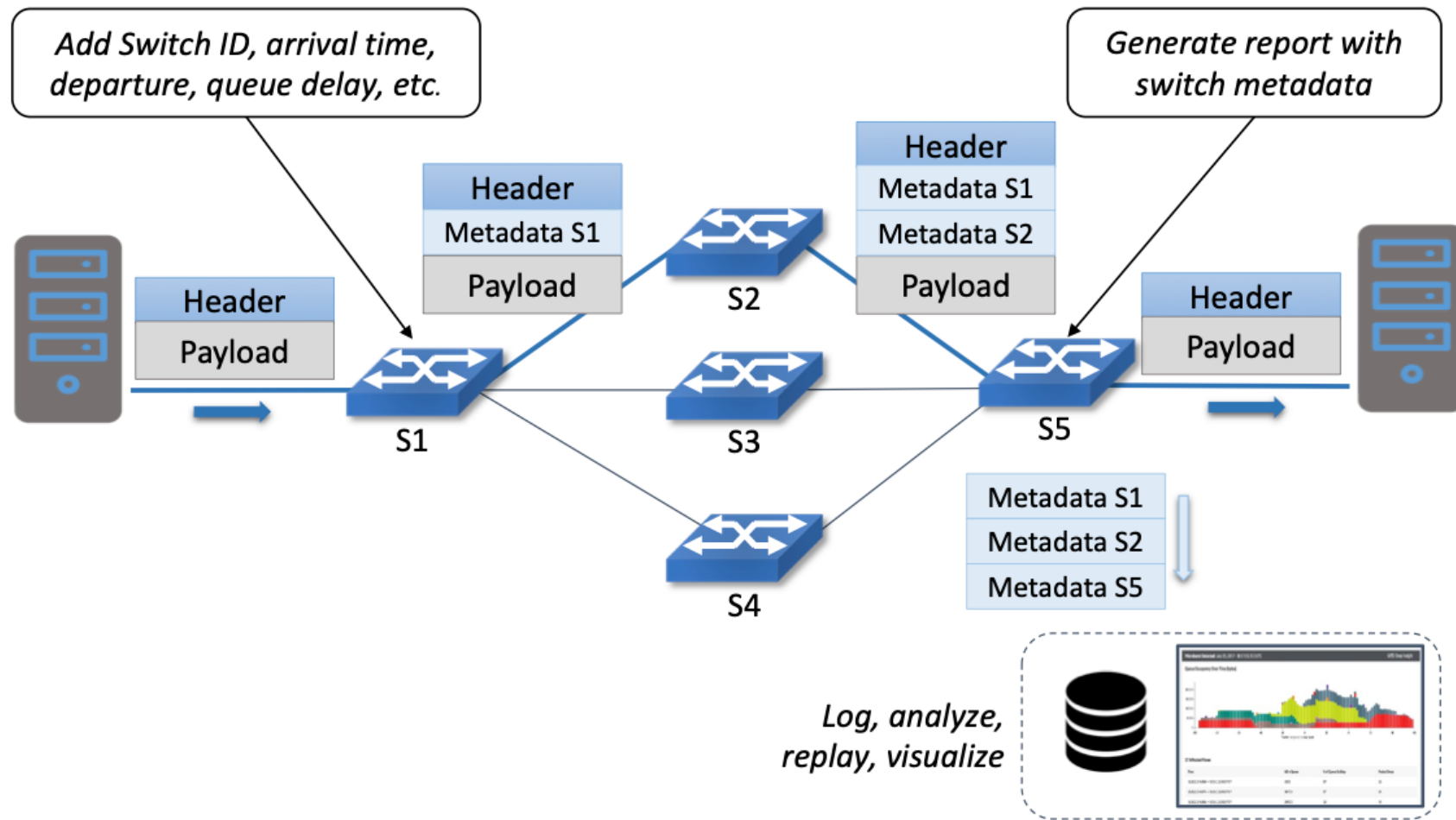
- **Redes de acesso**

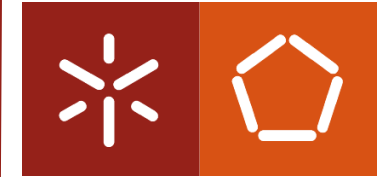
- Hardware específico das redes de fibra que pode ser substituído:
 - *OLT – Optical Line Terminators*
 - *BNG – Broadband Network Gateways*
- Hardware específico das redes de acesso sem fios
 - *NodeB* – a estação base da rede de acesso sem fios
 - EPC Enhanced Packet Core
- Operadores atacam estes segmentos que podem compensar monetariamente
 - Iniciativa designada por CORD “*Central-Office re-architected as a Datacenter*”
- As funcionalidades do hardware antigo podem ser implementadas por software nos *switches* do centro de dados
 - Por exemplo a funcionalidade dos BNG pode ser virtualizada, e correr numa VNF (*Virtual Network Function*), ou,
 - Implementada diretamente no *switch* – **VNF Offloading**
- Os progressos na desagregação da RAN (SD-RAN *Software defined radio Access Networks*) são mais lentos, mas estão em curso...

SDN: Casos de uso



- **Telemetria da Rede (um exemplo avançado)**

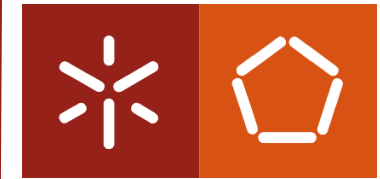




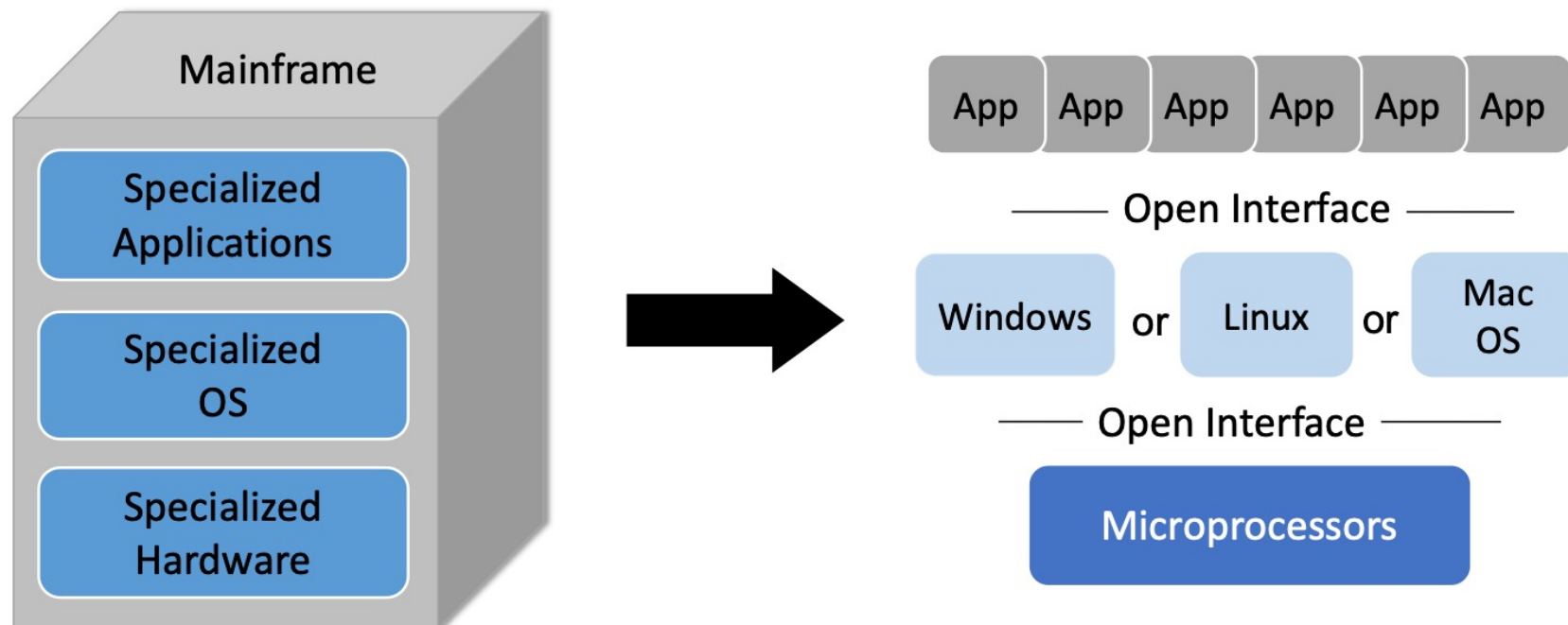
- **Telemetria da Rede (um exemplo avançado)**

- Designa-se por “**In-Band Network Telemetry**” - **INT**
- Só possível com a programação do processo de envio (*forwarding pipeline*)
- Contraste total com a monitorização da rede tradicional:
 - As estatísticas são colecionadas durante o processo de forwarding
 - Codificam-se instruções nos cabeçalhos dos pacotes
 - Essas instruções dizem aos swiches que informação de estado devem coletar
 - ... e como guardar essa informação nos cabeçalhos dos próprios pacotes
- Ainda num estado inicial e muito embrionário

SDN – Transformação do mercado



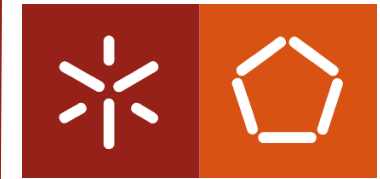
- **Analogia com o mercado de servidores**



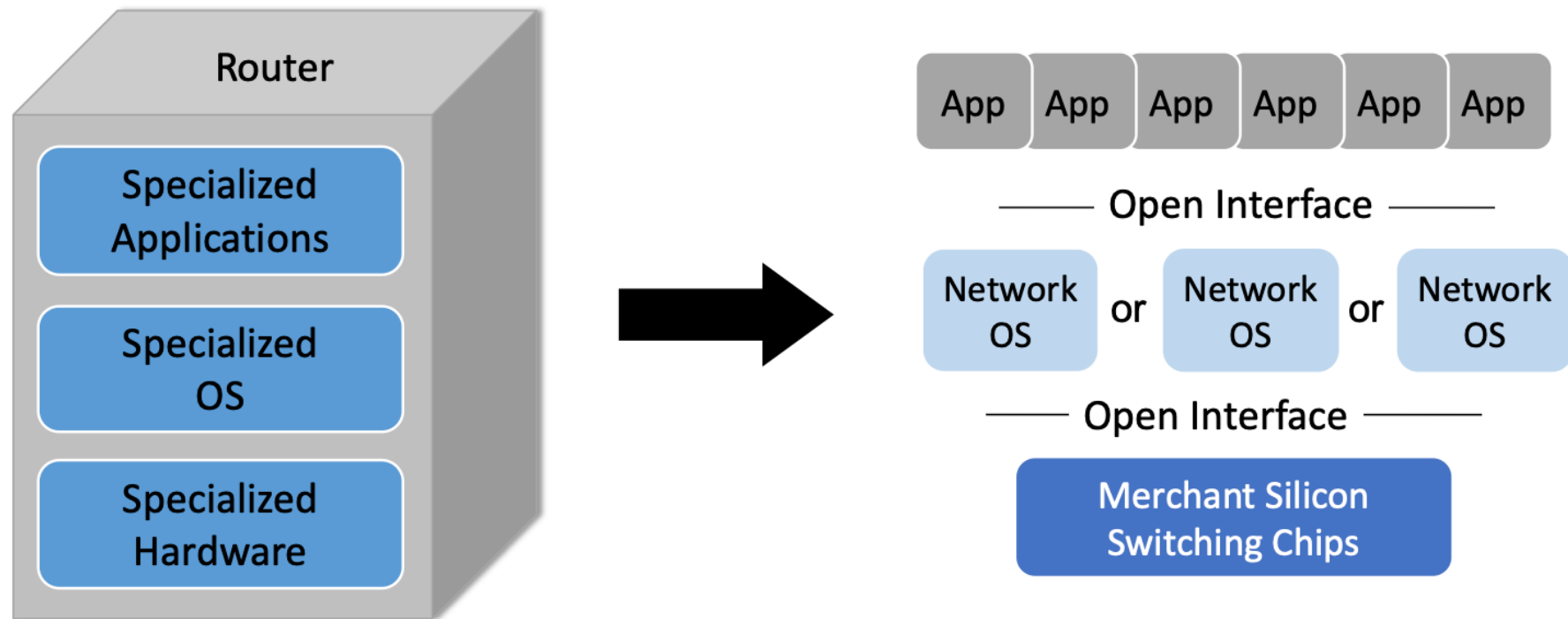
- Transformação do mercado vertical de mainframe num mercado horizontal com interfaces abertas e várias opções disponíveis em todos os níveis

<https://sdn.systemsapproach.org/intro.html>

SDN – Transformação do mercado

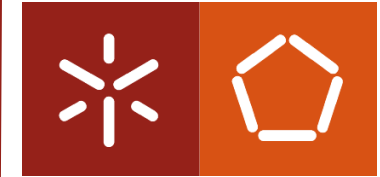


- **Mercado de equipamentos de rede (switches e routers)**



- Transformação do mercado vertical de routers num mercado horizontal com interfaces abertas e várias opções disponíveis em todos os níveis.

<https://sdn.systemsapproach.org/intro.html>



Parte 3 – Arquitetura

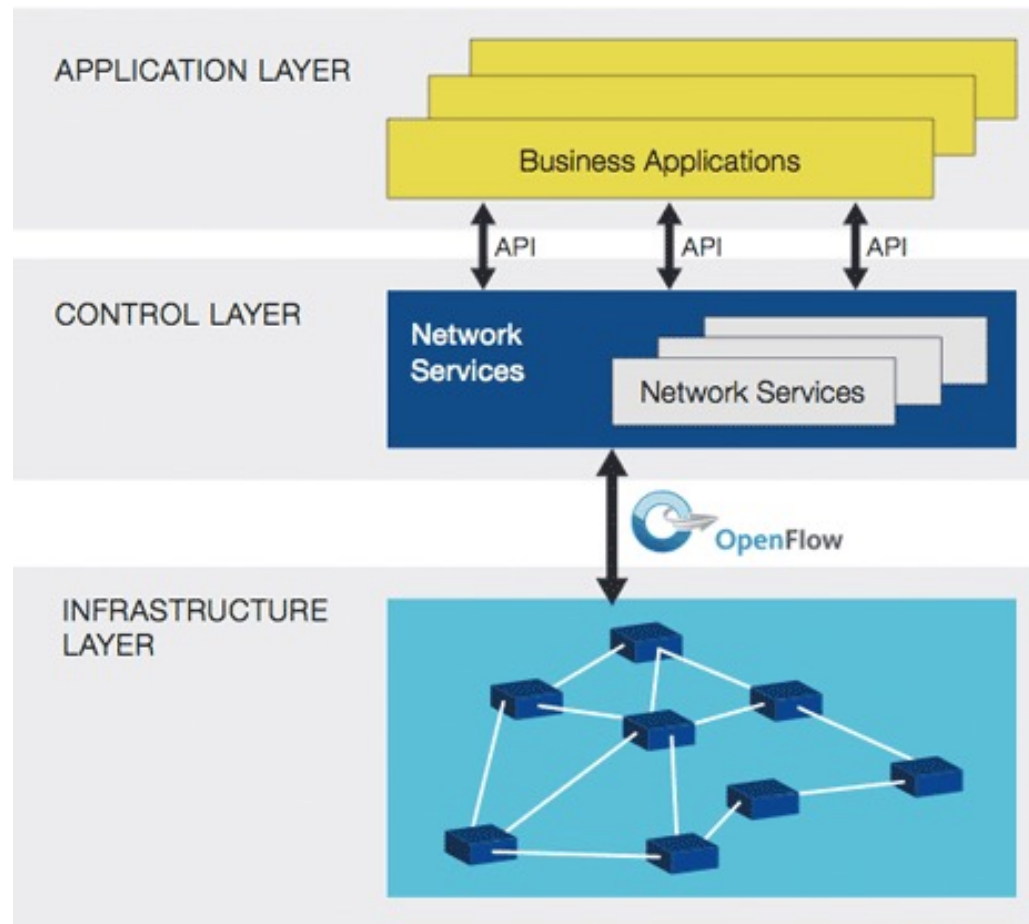
Baseado nos seguintes materiais:

- Capítulo 3 do livro “Software-Defined Networks: A Systems Approach”, Peterson, Cascone, O’Connor, Vachuska, and Davie (<https://sdn.systemsapproach.org/index.html>),

SDN – Arquitetura



- Perspetiva da **OpenNetworking.org** (**OpenFlow** Standard)

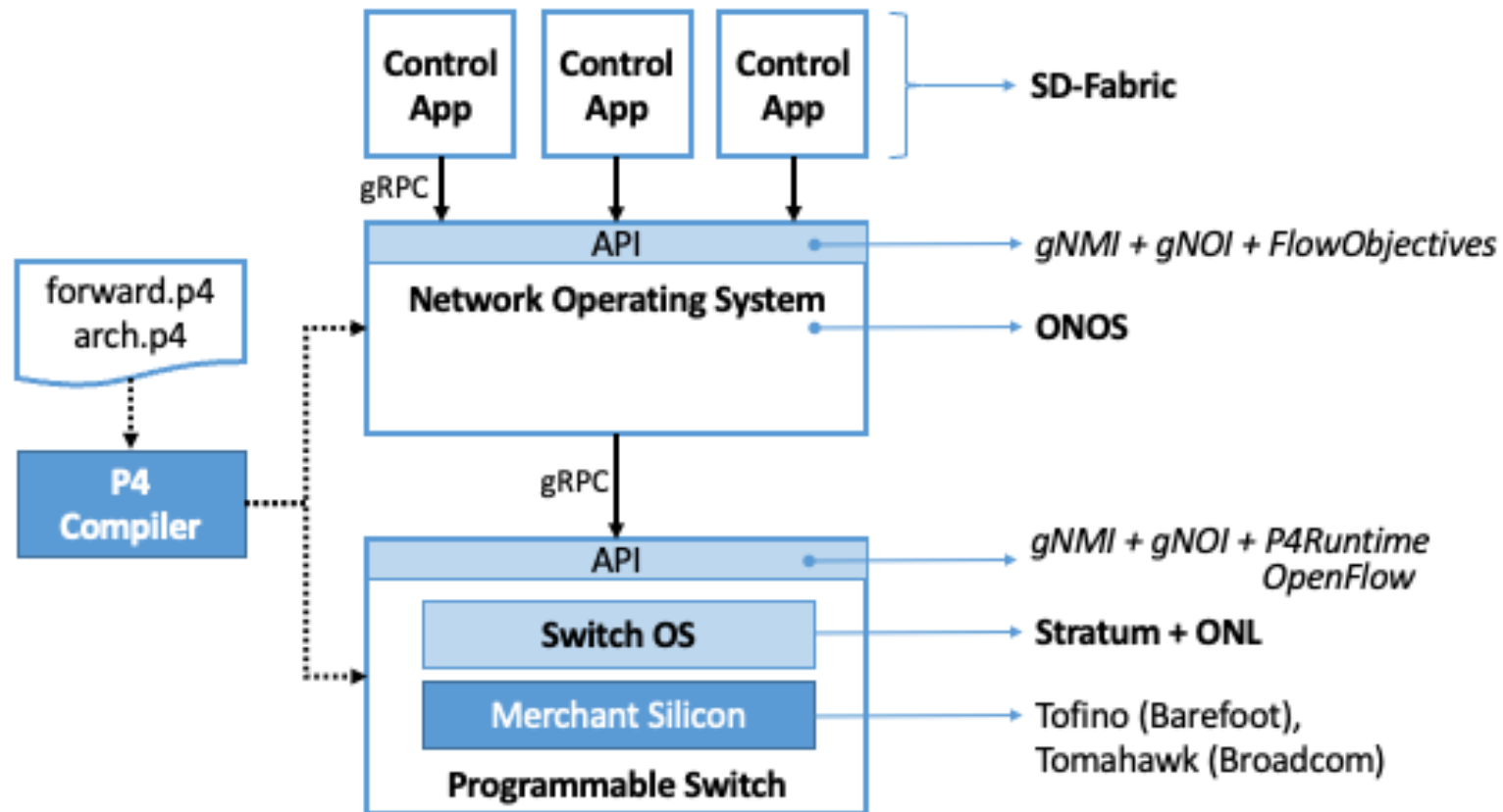


<https://opennetworking.org/sdn-resources/whitepapers/software-defined-networking-the-new-norm-for-networks/>

SDN – A pilha de software



- **Visão geral (outra?)**



Um passo à frente: Um **Bare-Metal Switch**, a correr um **Switch OS** local, e controlado por um **Network OS** global que hospeda um conjunto de **Aplicações de Controlo**

SDN – A pilha de software



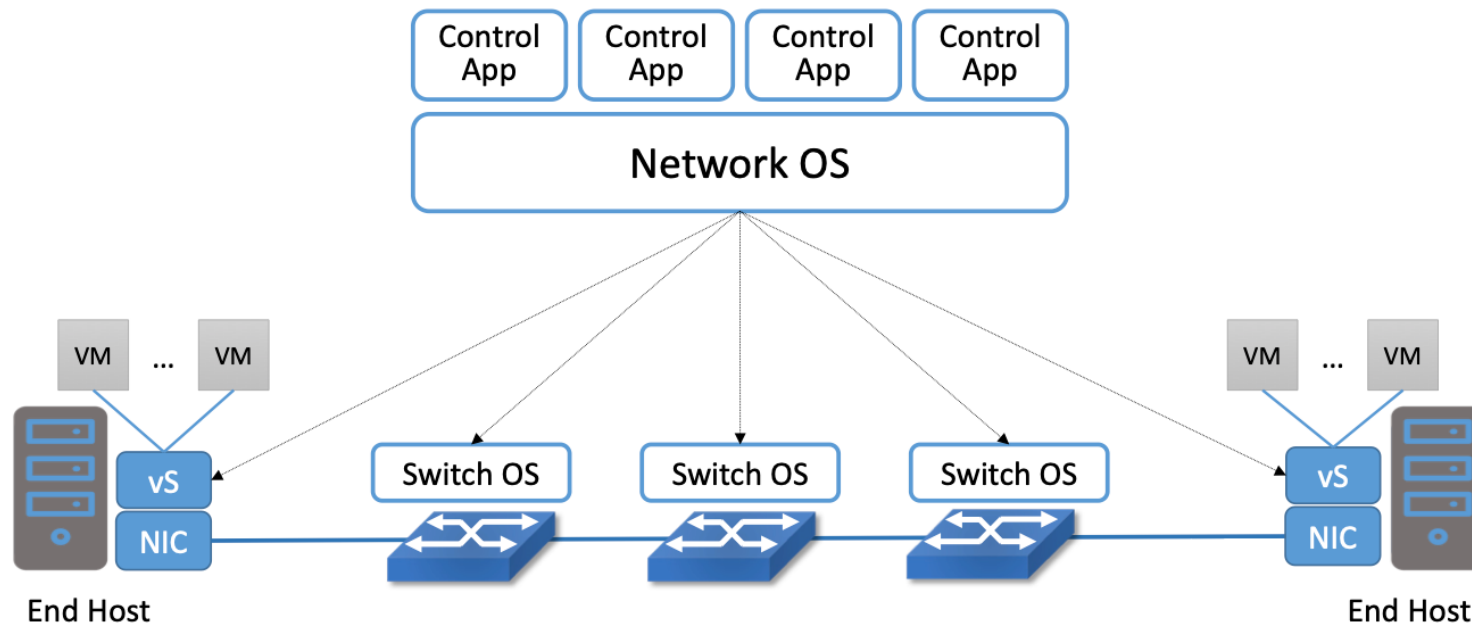
- **Visão geral**

- Um ponto a frisar: em todos os elementos desta visão, há projetos Open Source ativos a decorrer!
- Já foram testados em ambiente laboratorial e de produção todos os componentes
- Existem múltiplas alternativas, exemplos:
 - P4Runtime (como interface do switch) vs OpenFlow
 - gRPC/gNMI/gNOI vs NETCONF/YANG vs SNMP vs ...
- Arquitetura coerente sejam quais forem as escolhas
 - A ideia não é mostrar todas as alternativas exaustivamente
- **Esta visão mostra apenas a relação entre o SwitchOS e o NewtorkOS**

SDN – A pilha de software



- **Perspetiva da rede fim a fim: switch vs host**



- Dois aspetos muito importantes:
 - O **NewtorkOS** (por exemplo **ONOS**) atua na rede, extremo a extremo, enquanto o **SwitchOS** (ex: **Stratum**) só atua no contexto isolado do switch
 - A pilha SDN também corre nos sistemas finais (hosts) onde os **vswitches** ligam as máquinas virtuais (ou containers, ou...) entre si!

SDN – A pilha de software



- **Outros aspetos a considerar na figura anterior (perspetiva fim a fim):**

- Nos sistemas finais (*hosts*), tanto podemos ter **vSwitches** (ex: **Open vSwitch (OVS)**) como **Smart NICs** (*Smart Network Interface Card*) que ajudam ou substituem os vSwitches.

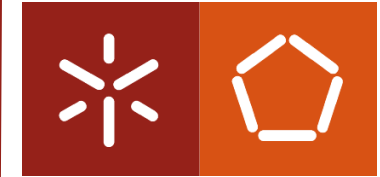
Open vSwitch (OVS) é um vSwitch de código aberto amplamente usado que oferece suporte ao OpenFlow como API norte.

- OVS aparece integrado com outras ferramentas como o DPDK (*Data Plane Development Kit*), também open source, para forwarding em processadores x86
 - Combina BPF (*extended Berkeley Packet Filter*) e XDP (*eXpress Data Path*)
 - Juntos oferecem suporte para programar regras do tipo “Match-Action” no Kernel
 - Semelhante ao espírito do OpenFlow e do P4, mas mais flexível, pois permite correr programas genéricos! OpenFlow define um conjunto restrito de ações e o P4 uma linguagem de programação limitada para exprimir ações



- **Hardware Switch aberto – “ferro rude” (*bare metal*)**

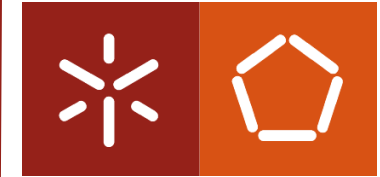
- Arquitetura agnóstica aos construtores de switches
- Mas que funciona por exemplo sobre
 - Switches *Tofino* (*Barefoot Networks*, agora do grupo *Intel*)
 - Implementa um pipeline de forwarding programável
 - Ou *Tomahawk* (*Broadcom*)
 - chip que implementa pipeline com conjunto fixo de funções
- Em ambos os casos, um par de programas em linguagem P4 definem o comportamento do pipeline de reenvio:
 - **forward.p4**: comportamento de forwarding
 - **arch.p4**: arquitetura lógica do chip de forwarding
 - Um compilador P4 gera código para correr tanto no switch como no NetworkOS
 - O componente do switch implementa o comportamento de forwarding, enquanto o componente do NetworkOS controla esse comportamento



Arquitetura SDN: *SwitchOS*

- **SwitchOS**

- Subindo uma camada na pilha: sistema operativo do *switch*, que corre num processador genérico incluído no hardware do *switch*
- Responsável por suportar a API que trata pedidos vindos do **NetworkOS**
- Implica atuar internamente no pipeline de reenvio de pacotes
- Existem vários **SwitchOS** *open source*, como por exemplo o **SONiC** (desenvolvido na Microsoft Azure)
- **Vamos focar atenção no Stratum + ONL**
 - **Open Network Linux (ONL)**: distribuição Linux para Switches, preparada pela empresa *Big Switch Networks*
 - **Stratum**: também designado por *Thin Switch OS*, preparado pela Google, com função de mapear/traduzir os pedidos da API nos recursos internos do switch
- *Stratum* media todas as interações do Switch com o mundo exterior
 - Incluindo carregar o código compilado P4 que estabelece o contrato entre o plano de dados e o plano de controlo
 - Substitui o OpenFlow e as suas regras, por uma especificação própria



Arquitetura SDN: *SwitchOS*

- **SwitchOS: API de Gestão**

- **P4Runtime:**

- um interface para controlar o comportamento de forwarding em tempo de execução
- permite popular as tabelas de *forwarding* e manipular o estado de forwarding
- independente de qualquer programa P4 e agnóstico em relação ao hardware
- contrasta com a natureza prescritiva do OpenFlow (que é alternativa)

- **gNMI (gRPC Network Management Interface):**

- Usado para obter ou alterar (*get/set*) estado da configuração
- Normalmente emparelhado com **modelos YANG** que descrevem a estrutura dos dados de configuração e a sua organização em árvore

- **gNOI (gRPC Network Operations Interfaces):**

- Usado para obter ou alterar (*get/set*) do estado operacional
- Exemplo: gestão de certificados, teste dos dispositivos, *upgrade* do *software*, resolução de problemas de rede, etc.

Arquitetura SDN: *NetworkOS*



- **NetworkOS:**

- Plataforma de gestão e configuração de uma rede: controlador SDN centralizado
- Responsável por monitorizar o estado dos switches, detetar falhas nas portas ou nos links
- Mantem uma visão topológica global da rede, de acordo com o estado atual
- Disponibiliza essa visão da rede às aplicações de controlo!
- As aplicações de controlo dão instruções ao **NetworkOS** para controlar os fluxos de dados nos equipamentos de acordo com o serviço que pretendem disponibilizar

A forma como as “*instruções*” são expressas é um aspecto chave da **API do NetworkOS**

- Um sistema operativo específico é o **ONOS (Open Network Operating System)**
- Uma referência em termos de desempenho, escalabilidade e disponibilidade

Arquitetura SDN: *NetworkOS*



- **NetworkOS: ONOS**

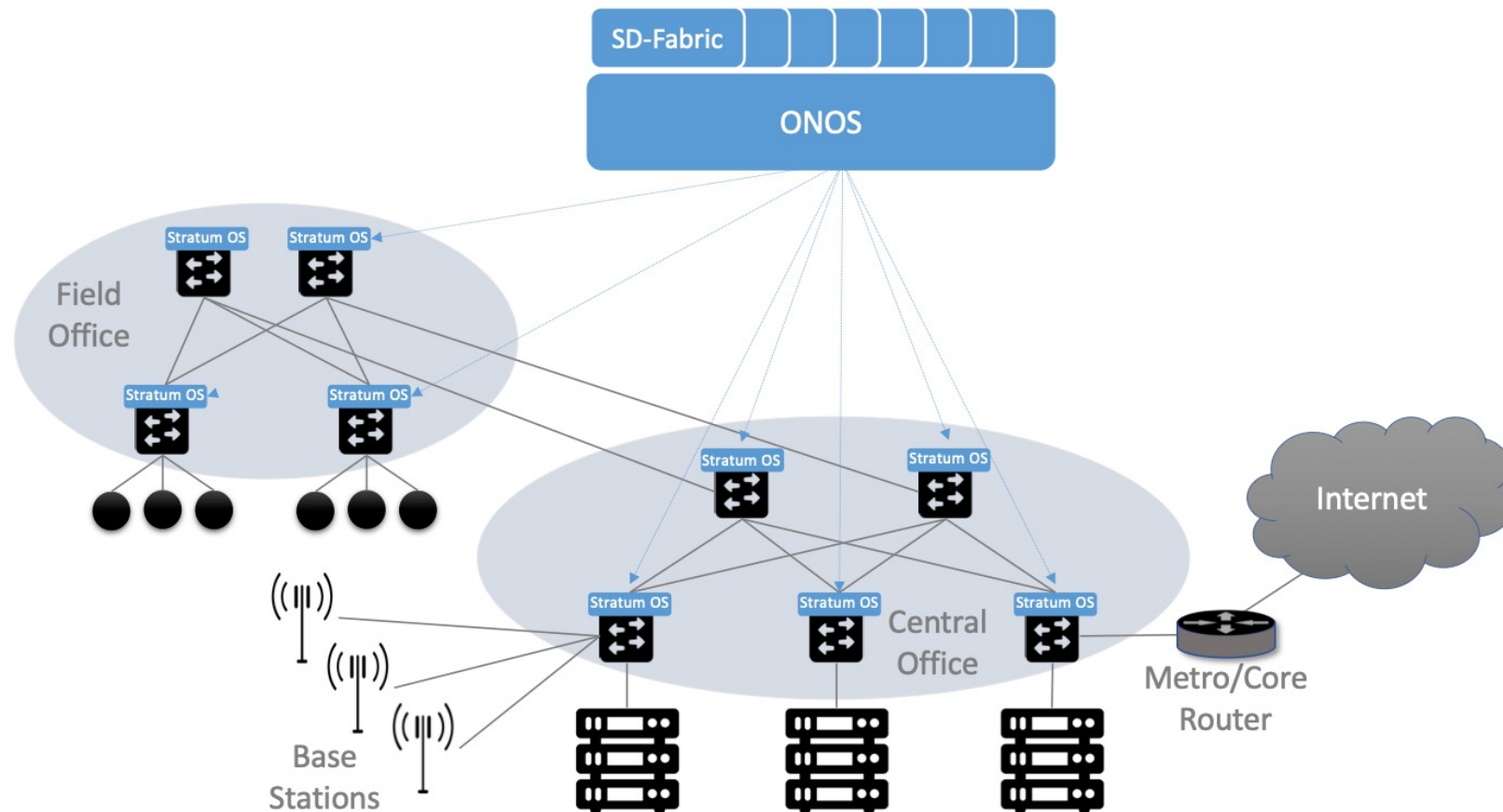
- O ONOS é responsável, a alto nível, por:
 - **Gerir a topologia:** regista um inventário topológico completo para partilhar
 - **Gerir a configuração:** facilita a manutenção das configurações, voltar atrás (*roll back*), validação, espelha a configuração dos dispositivos de rede
 - **Controlar os switches:** controla o plano de dados dos switches, regras de fluxos, grupos, métricas, etc.
- ONOS usa na API norte a abstração FlowObjectives, que generaliza as regras de fluxos de forma independente do pipeline
- Tal como acontece num sistema operativo convencional, as aplicações desenvolvidas para o ONOS não são fáceis de portar para outros sistemas operativos de rede
- O que se pretende é a que a API seja aberta e esteja bem definida!

Um aspeto chave é o armazenamento escalável *key / value*! A manutenção de informação de estado de forma eficiente e escalável. ONOS usa o Atomix.

Arquitetura SDN: *Apps de controlo*



- **Exemplo: SD fabric**



- Uma suite de apps de controlo: VLAN e bridging L2, IPv4 e IPv6 unicast e multicast routing, DHCP L3 relay, Dual-homing de servidores e routers, QinQ forwarding, MPLS-based pseudo-wires